



TUCAM-API 开发指南 Python



鑫图光电有限公司

保留所有的权利

目录

1. 使用前阅读	4
2. 简介	4
3. 概述	5
3.1 层结构	5
3.2 原理	5
3.3 规则	6
3.3.1 句柄相关的内容	6
3.3.2 函数相关内容	6
3.3.3 性能相关内容	7
3.3.4 属性相关的内容	7
3.3.5 触发相关的内容	8
4. 编程指引	10
4.1 搭建编程环境	10
4.1.1 pycharm64 2022.1 开发环境配置	10
4.2 快速上手	12
4.2.1 起始和终止	13
4.2.2 属性获取和设置	15
4.2.3 属性获取和设置	17
4.2.4 内存管理	19
4.2.5 捕获控制	22
4.2.6 文件管理	23
4.2.7 扩展控制	27
4.2.8 其它	28
5. 参考	31
5.1 类型和常量	31
5.1.1 TUCAMRET 错误代码	31
5.1.2 TUCAM_IDINFO 产品信息代码	33

5.1.3 TUCAM_IDCAPA 性能代码	34
5.1.4 TUCAM_IDPROP 属性代码	36
5.1.5 TUCAM_IDPPROP 图像拼接代码	37
5.1.6 TUCAM_IDCROI 区域计算代码	38
5.1.7 TUCAM_CAPTURE_MODES 捕获模式代码	38
5.1.8 TUIMG_FORMATS 图像格式代码	38
5.1.9 TUCAM_TRIGGER_EXP 触发曝光时间模式代码	39
5.1.10 TUCAM_TRIGGER_EDGE 激发边沿类型代码	39
5.1.11 TUCAM_TRIGGER_READOUTDIRRESET 触发读取输出复位代码	39
5.1.12 TUCAM_TRIGGER_READOUTDIR 触发输出方向代码	39
5.1.13 TUFRM_FORMATS 帧格式代码	39
5.1.14 TUCAM_OUTPUTTRG_PORT 触发输出端口代码	40
5.1.15 TUCAM_OUTPUTTRG_KIND 触发输出种类代码	40
5.1.16 TUCAM_OUTPUTTRG_EDGE 触发输出边沿代码	40
5.2 结构体	40
5.2.1 初始化	40
5.2.2 打开相机	41
5.2.3 打开图像	41
5.2.4 相机信息	42
5.2.5 性能的属性	42
5.2.6 属性的属性	43
5.2.7 性能 / 属性值文本	43
5.2.8 ROI 的属性	44
5.2.9 区域计算属性	44
5.2.10 触发的属性	45
5.2.11 触发输出的属性	46
5.2.12 任意 Bin 属性	46
5.2.13 帧结构	47
5.2.14 文件保存	48
5.2.15 录像保存	48
5.2.16 寄存器读写	49
5.3 函数	49



5.3.1 API 初始化 / 反初始化	49
5.3.2 打开、关闭相机	50
5.3.3 性能获取和设置	52
5.3.4 属性获取和设置	55
5.3.5 内存管理	57
5.3.6 捕获控制	62
5.3.7 文件管理	69
5.3.8 扩展控制	71
6. 常见问题解答 (FAQ)	72
6.1 问题排查思路	72
6.2 常见问题解决方法	72

1. 使用前阅读

这份文档和软件示例代码是 TUCSEN 的内部文件和公布内容，以使用户能够创建使用 TUCSEN 数字相机中的应用。本文档和软件示例代码只针对上述目的而公开的，并且不构成所有者的许可、转让或任何其他权力。使用软件文档的所有风险和结果仍然取决于用户。

使用软件文档的所有风险和结果仍然取决于用户。本文档可能包括技术错误或印刷错误。并且不能保证这样的错误或文本所产生的任何损害。TUCSEN 不承诺更新或保持当前的这个文档中所包含的信息。

所有品牌和产品名称都是其各自所有者的商标或注册商标。TUCSEN 对文档的版权保留所有权利。在没有 TUCSEN 的事先书面许可下，文档的任何部分不得被复制、传递、转录，存储在检索系统或翻译成任何语言或计算机语言，以任何的形式，或以任何方式，任何的手段如：电子、机械、电磁、光学、化学手动或其他。

2. 简介

TUCAM-API

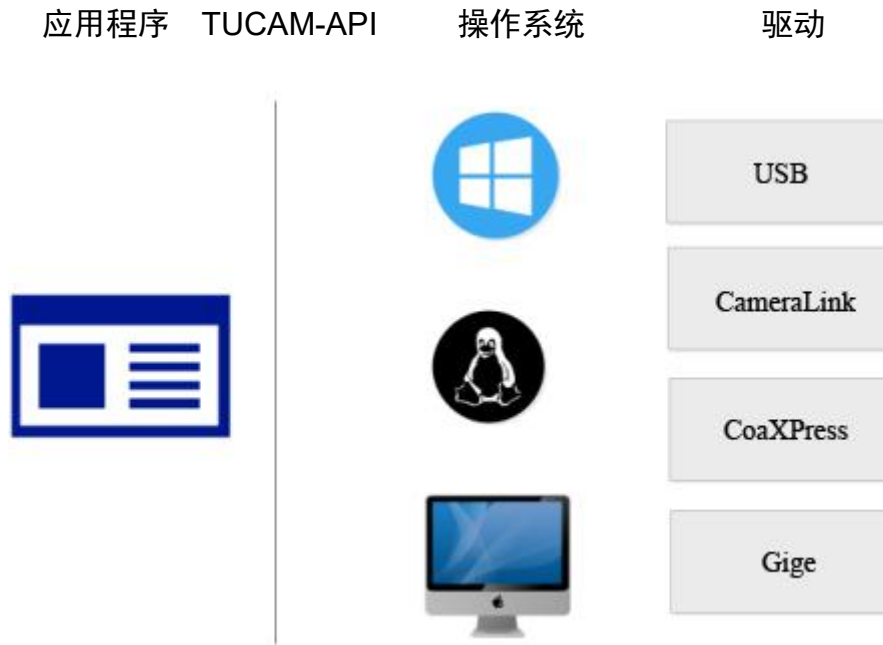
TUCAM-API 是用于控制 TUCSEN 制造的相机的应用软件程序接口。TUCAM-API 目前支持连接 USB 、 CameraLink 、 CoaXPress 、 Gige 数据接口，在初始化时自动进行匹配工作。另外 API 设计特别容易理解。出于这个原因，函数接口的数量限制到最少，并且函数的调用格式采用类似 C 语言调用接口写法。

说明

部分扩展的函数是某些特定数字相机可以使用的附加功能。不同数字相机的数值可能不同，这取决于捕捉图像所使用的数字相机型号。数值应该简单地视为指南，而不是精确值。具体可以参考系列相机性能、属性文件。

3. 概述

3.1 层结构



TUCSEN 数字相机通过 TUCam.py 调用 SDK 连接不同的操作系统的数字相机驱动来达到控制数字相机和采集图像数据的作用。

3.2 原理

数字相机的具体总线接口和库通过 TUCam.py 进行接口转换。您只需要访问 TUCam.py 层。模块层提供更高級的 TUCAM-API 集成。模块可以不断更新访问新相机和提供新接口技术。

TUCAM-API 不包含用于显示图像的程序。因为一些显示图像的方法难以预测，这取决于应用程序，它是不可能支持所有这些通用模块的。当调用显示程序时，检测图像是否更新，并且在更新后绘制图像。对于更详细的信息，请参阅示例源代码。

3.3 规则

3.3.1 句柄相关的内容

名称	描述
hIdxTUCam	是 API 上指定目标相机的句柄，几乎所有的 API 都需要这个句柄作为参数。 TUCAM_Dev_Open 函数提供这个句柄，利用 TUCAM_Dev_Release 函数释放句柄，终止设备使用。
hIdxTUImg	是 API 指定打开图片的句柄。 TUIMG_File_Open 函数提供这个句柄，利用 TUCAM_File_Close 关闭句柄，释放图片资源。

3.3.2 函数相关内容

函数名称

所有 TUCAM-API 函数都以大写字母“TUCAM_”作为前缀。TUCAM-API 所有函数按照性能被归类，同组的函数有相同前缀。以下是函数前缀列表：

函数前缀	函数名称	说明
TUCAM_Api_	Init, Uninit	初始化、反初始化 API
TUCAM_Dev_	Open, Close, GetInfo, GetInfoEx	打开、关闭相机、获取信息
TUCAM_Capa_	GetAttr, GetValue, SetValue, GetValueText	性能获取和设置
TUCAM_Prop_	GetAttr, GetValue, SetValue, GetValueText	属性获取和设置
TUCAM_Buf_	Alloc, Release, AbortWait, WaitForFrame, CopyFrame	内存管理相关控制
TUCAM_Cap_	SetROI, GetROI, SetTrigge, GetTrigger DoSoftwareTrigger, SetTriggerOut, GetTriggerOut, Start, Stop	图像数据捕获相关控制
TUCAM_File_	SaveImage, LoadProfiles, SaveProfiles, Open, Close	图片文件、配置文件相关操作
TUCAM_Rec_	Start, AppendFrame, Stop	视频文件保存相关接口
TUCAM_Reg_	Read, Write	相机相关配置寄存器读/写
TUCAM_Draw_	Init, Frame, UnInit	绘制图像接口（Windows 系统）
TUCAM_Calc_	SetROI, GetROI	相关功能 ROI 计算的区域读/写
TUCAM_Vendor_	Config, ConfigEx, Update, Prop_GetAttr Prop_SetValue, Prop_GetValue, Prop_GetValueText, ResetIndexFrame, WaitForIndexFrame, Buf_Attach, Buf_Detach	厂商控制的相关接口

函数参数

除了 TUCAM_Api_Init 初始化、TUCAM_Api_Uninit 反初始化、TUCAM_Dev_Open 打开相机等函数，几乎所有的 TUCAM-API 都需要 hIdxTUCam 句柄作为第一个参数。一些函数还需要一个指向结构体的指针作为参数，在结构体参数中有 [in] 标识变量，意味着应用程序必须在调用之前设置值，有 [out] 标识变量，意味着函数返回时填充一个值。

函数返回

TUCAM-API 所有的函数执行后都有返回一个 TUCAMRET 值。所有的返回结果都被定义成 TUCAMRET_ 开头的。如果函数执行成功返回 TUCAMRET_SUCCESS，在开发中建议您在执行完函数后，判断返回结果值为 TUCAMRET_SUCCESS 再进行下一步操作，否则给出错误提醒，方便问题的查找。

3.3.3 性能相关内容

在 TUCAM 中什么是性能？

“性能”是指设备支持能力。TUCAM-API 中定义了 TUCAM_IDCAPA 性能常量，每个性能都有个唯一 ID 以 TUIDC_ 为前缀。用户通过带有 TUCAM_CAPA_ 前缀的函数查询设备是否支持某项能力，对性能值进行获取、设置。每个性能值都被处理成整型，每个性能都有自己 ID，最小值，最大值，默认值。

多性能值

一些性能可能有多个值，它们的值有关联的文本，我们称这样的文本为值文本。通过 TUCAM_Capa_GetValueText 函数获取值文本。

3.3.4 属性相关的内容

在 TUCAM 中什么是属性？

“属性”是指设备参数。TUCAM-API 中定义了 TUCAM_IDPROP 常量，每个属性都有个唯一 ID 以 TUIDP_ 为前缀。用户通过 TUCAM_Prop_ 前缀的函数获取、设置、

查询设备参数。每个属性值都被处理成双精度浮点型，每个属性都有自己 ID,最小值，最大值，默认值。

多属性值

一些属性可能有多个值，它们的值也有关联的文本，我们称这样的文本为值文本。通过 TUCAM_Prop_GetValueText 函数获取。

3.3.5 触发相关的内容

捕获模式：

相机的捕获模式分为以下 2 类：

- 1) 序列模式（流模式）：用来捕获连续的图像数据。
- 2) 触发模式：相机通过内、外部信号来捕获图像。我们称这个选项为“触发模式”，您可以调用 TUCAM_Cap_SetTrigger 来配置此选项。我们也把外部信号称为“外部触发”。

图像单元：

通常情况下是二维的，具有垂直和水平方向。

帧：是一个用于图像数据的单位。对于一帧，一个像素的数据是从左到右和从上到下对齐的。这是一系列的图像数据单位。

触发属性：

触发属性设置，参考 TUCAM_TRIGGER_ATTR 结构体。包括触发模式、曝光模式、激发电平类型、触发延迟、输出帧数。不同触发模式支持的参数不一样，例如软件触发，就不支持设置边沿激发类型，触发曝光时间由软件设定（软触发没有上升沿、下降沿）。

模式	触 发 曝 光 模 式 (TUCAM_TRIGGER_EXP)	边 沿 激 发 类 型 (TUCAM_TRIGGER_EDGE)	延时	帧数
标准触发	支持	支持	支持	支持
同步触发	支持	支持	不支持	不支持

全局触发	不支持	支持	不支持	不支持
软件触发	不支持	不支持	不支持	不支持

触发模式

- 1) 标准模式（Standard）：当相机接收到电平信号后（由激活边沿决定）开始进行一帧或多帧的图像捕获，捕获帧数由配置参数决定。
- 2) 同步模式（Synchronization）：当相机接收到电平信号后（由激活边沿决定）开始进行曝光，当收到相反的电平信号后，结束曝光、并且进行图像数据的捕获。即实现每一帧的曝光与读出，均与外触发信号完全同步。
- 3) 全局模式（Global）：在相机未触发前进行预触发。当相机接收到电平信号后（由激活边沿决定）或者为软件设定的曝光时间时，结束当前正在进行的重置操作，待曝光结束时捕获图像数据，并重新开始预触发。该方式用于控制卷帘曝光模式的相机实现全局曝光模式。
- 4) 软件触发（Software）：通过软件下发命令模拟触发信号。

曝光模式

- 1) 曝光时间：接收到触发信号后，由 TUIDP_EXPOSURETM 设置的曝光时间决定。
- 2) 电平宽度：接收到触发信号后，曝光时间由电平的宽度所决定。

注：标准模式（Standard）和全局模式（Global）可配置这两个选项。同步模式（Synchronization）只能是电平宽度。软件触发只能由软件设置的曝光时间决定。

激发电平类型

- 1) 上升电平（Rising Edge）：接收到的触发电平处于上升沿时开始曝光。
- 2) 下降电平（Falling Edge）：接收到的触发电平处于下降沿时开始曝光。

触发延迟

- 1) 输入：表示接收到一个触发信号后，可以设置多长的延迟时间才使相机进行触发曝光。

- 2) 输出：表示接收到一个输出信号后，可以设置多长的延迟时间才使相机电平信号输出。

触发输出端口

触发输出信号有 3 个端口 TRIG.OUT1、TRIG.OUT2、TRIG.OUT3，对应 Port1、Port2、Port3。

三种输出信号默认是始终开启状态，相机将电平信号输出给第三方设备作为其输入信号。三种信号可以独立工作、同时输出到不同设备。

触发输出硬件接口有 3 个管脚，可以对 3 个端口进行配置，并且 3 个端口互不干涉。

触发输出电平

- 1) High：始终以高电平信号输出。
- 2) Low：始终以低电平信号输出。
- 3) Exposure Start：以第一行开始曝光到最后一行开始曝光的指示电平为信号输出。
- 4) Readout End：以第一行开始读出到最后一行开始读出的电平信号输出。
- 5) Global Exposure：以最后一行开始曝光至第一行结束曝光（所有行处于曝光状态）的指示电平信号输出。

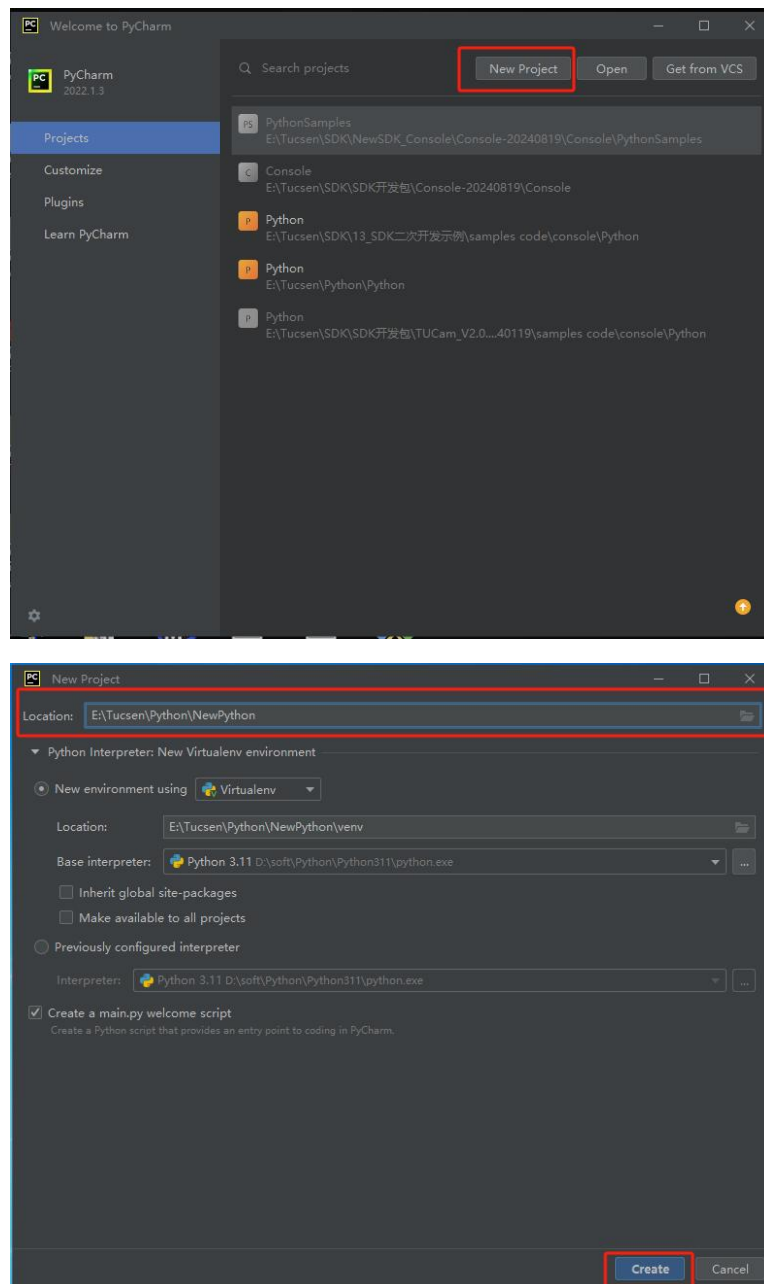
4. 编程指引

4.1 搭建编程环境

4.1.1 pycharm64 2022.1 开发环境配置

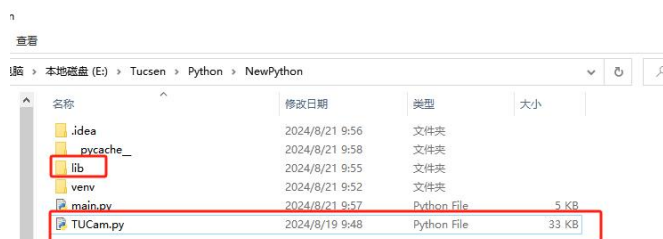
1) 新建工程

打开 pycharm64 软件，点击“New Project”后，对话框中点击“Create”：



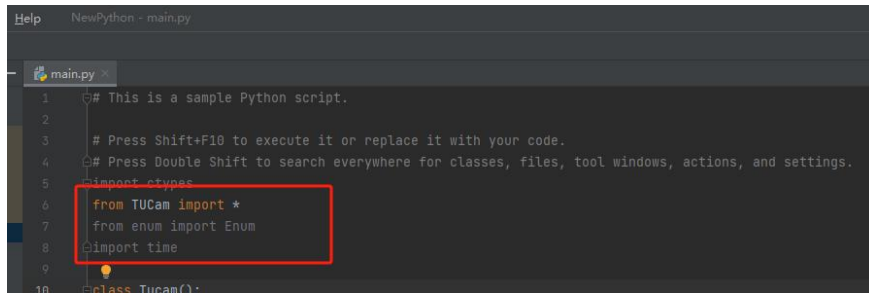
2) 配置引用

拷贝我们示例代码中的 TUCam.py 和 lib 文件夹到新建的目录下。



3) 配置引用文件 TUCam

如图所示：

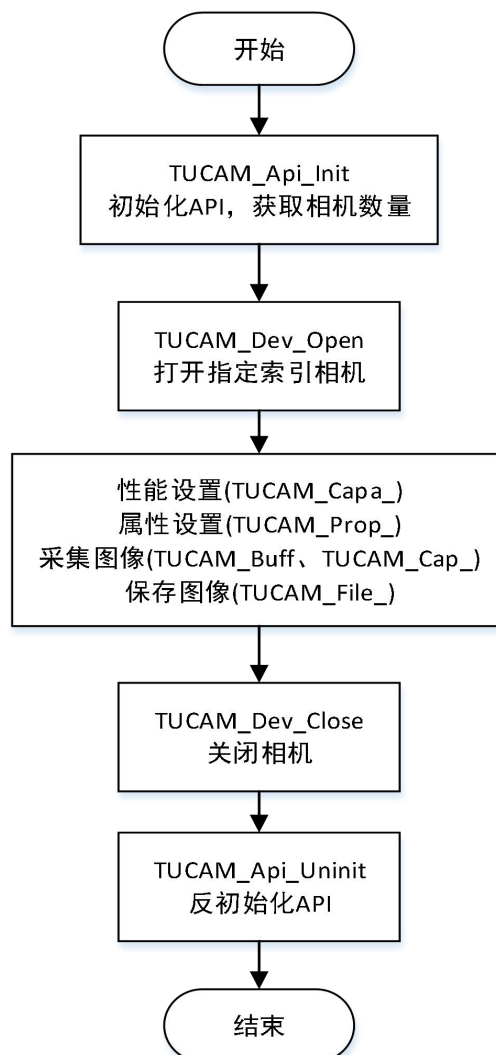


```
1  # This is a sample Python script.
2
3  # Press Shift+F10 to execute it or replace it with your code.
4  # Press Double Shift to search everywhere for classes, files, tool windows, actions, and settings.
5  import ctypes
6  from TUCam import *
7  from enum import Enum
8  import time
9
10 class TUCam():
```

注意：根据用户应用选择适合（x86、x64）库引用，在 TUCam.py 文件上可做切换。

4.2 快速上手

TUCAM-API 工作调用流程：



4.2.1 起始和终止

4.2.1.1 调用顺序

首先，驱动程序初始化。当应用程序执行设备句柄的初始化已经成功完成，获取可以控制的相机数量。

其次，当应用程序启动时，调用 SDK 的 API 初始化函数执行初始化操作。

再次，当初始化函数调用成功后，其他函数才能正常被调用执行。

相机终止函数用于程序的关闭。当一个相机被挂起，或资源被释放而不再控制相机时执行的函数（例如，当应用程序退出）。当终止函数被调用时，其他的功能函数调用将不被执行，直到初始化函数再次调用之后。

4.2.1.2 SDK 的 API 初始化

API 使用 TUCAM_Api_Init 函数进行初始操作。该函数初始化帧采集和控制相机。

[接口参考 5.3.1](#)

4.2.1.3 相机初始化

相机初始化使用 TUCAM_Dev_Open 函数。该函数获取必要的相机句柄 HDTUCAM 来做为其他函数的输入参数。[接口参考 5.3.2](#)

4.2.1.4 相机产品信息

当调用 TUCAM_Dev_Open 函数打开相机之后，可以通过相机句柄获取相机的产品信息。例如，相机型号、固件版本等。[接口参考 5.3.2](#)，产品信息参考 [TUCAM_IDINFO](#)

4.2.1.5 终止程序

终止相机程序使用 TUCAM_Dev_Close 函数。调用这个函数释放被用于相机帧获取的端口及资源。这个函数被调用后，相机将不再被控制。

4.2.1.6 示例代码

SDK 的 API 初始化参考以下代码，或者参考 Samples 目录 00_init_open.py 工程，相机产品信息参考 samples code 目录 01_get_info.py/get_infoex.py 工程。

```
1. class Tucam():
2.     def __init__(self):
3.         self.Path = './'
4.         self.TUCAMINIT = TUCAM_INIT(0, self.Path.encode('utf-8'))
5.         self.TUCAMOPEN = TUCAM_OPEN(0, 0)
6.         # ch:初始化相机枚举相机个数
7.         TUCAM_Api_Init(pointer(self.TUCAMINIT))
8.         print(self.TUCAMINIT.uiCamCount)
9.         print(self.TUCAMINIT.pstrConfigPath)
10.        print('Connect %d camera' %self.TUCAMINIT.uiCamCount)
11.
12.    def OpenCamera(self, ldx):
13.        if ldx >= self.TUCAMINIT.uiCamCount:
14.            return
15.        self.TUCAMOPEN = TUCAM_OPEN(ldx, 0)
16.        # ch:打开相机 ldx
17.        TUCAM_Dev_Open(pointer(self.TUCAMOPEN))
18.        if 0 == self.TUCAMOPEN.hIdxTUCam:
19.            print('Open the camera failure!')
20.            return
21.        else:
22.            print('Open the camera success!')
23.
24.    def CloseCamera(self):
25.        # ch:关闭相机 | en:Close camera
26.        if 0 != self.TUCAMOPEN.hIdxTUCam:
27.            TUCAM_Dev_Close(self.TUCAMOPEN.hIdxTUCam)
28.            print('Close the camera success')
29.
30.    def UnInitApi(self):
31.        # ch:反初始化相机 | en:Uninitial Cameras
32.        TUCAM_Api_Uninit()
33.
34.    if __name__ == '__main__':
35.        demo = Tucam()
36.        demo.OpenCamera(0)
37.        if demo.TUCAMOPEN.hIdxTUCam != 0:
```

```
38.         demo.CloseCamera()  
39.         demo.UnInitApi()
```

4.2.2 属性获取和设置

4.2.2.1 调用顺序

1) 获取性能属性

利用 TUCAM_Capa_GetAttr 函数，通过设置性能 ID 获取性能属性，其中包括最大值、最小值、默认值。

2) 获取、设置性能值

通过 TUCAM_Capa_GetValue 和 TUCAM_Capa_SetValue 函数来设置和获取性能值。所有的相机性能值都被处理成一个整型，某些性能具有多个性能值。设置和获取通常在相机进行捕获之前或者之后已经完成。如果设置函数在数据捕获时被调用，在某些情况下可能会返回错误的代码 TUCAMRET 。[接口参考 5.3.3](#)

3) 获取性能值文本

TUCAM_Capa_GetValueText 函数获取性能值文本，用于描述每个性能值具体含义。

4.2.2.2 性能索引

每个性能都有唯一的 ID，参考 [TUCAM_IDCAPA](#)

注意：如果相机不支持该性能 ID，将返回错误代码 TUCAMRET_NOT_SUPPORT 或 TUCAMRET_INVALID_IDCAPA。

4.2.2.3 示例代码

参考以下代码或 samples code 目录 23_set_resolution.py 工程：

```
1. class Tucam():  
2.     def __init__(self):
```



```
3.         self.Path = './'
4.         self.TUCAMINIT = TUCAM_INIT(0, self.Path.encode('utf-8'))
5.         self.TUCAMOPEN = TUCAM_OPEN(0, 0)
6.         TUCAM_Api_Init(pointer(self.TUCAMINIT))
7.         print(self.TUCAMINIT.uiCamCount)
8.         print(self.TUCAMINIT.pstrConfigPath)
9.         print('Connect %d camera' %self.TUCAMINIT.uiCamCount)
10.
11.     def OpenCamera(self, ldx):
12.         if ldx >= self.TUCAMINIT.uiCamCount:
13.             return
14.         self.TUCAMOPEN = TUCAM_OPEN(ldx, 0)
15.         TUCAM_Dev_Open(pointer(self.TUCAMOPEN))
16.         if 0 == self.TUCAMOPEN.hIdxTUCam:
17.             print('Open the camera failure!')
18.             return
19.         else:
20.             print('Open the camera success!')
21.
22.     def CloseCamera(self):
23.         if 0 != self.TUCAMOPEN.hIdxTUCam:
24.             TUCAM_Dev_Close(self.TUCAMOPEN.hIdxTUCam)
25.             print('Close the camera success')
26.
27.     def UnInitApi(self):
28.         TUCAM_Api_Uninit()
29.
30.     def SetResolution(self):
31.         capa = TUCAM_CAPA_ATTR()
32.         capa.idCapa = TUCAM_IDCAPA.TUIDC_RESOLUTION.value
33.         TUCAM_Capa_GetAttr(self.TUCAMOPEN.hIdxTUCam, pointer(capa))
34.         count = capa.nValMax - capa.nValMin + 1
35.         for i in range(count):
36.             TUCAM_Capa_SetValue(self.TUCAMOPEN.hIdxTUCam, capa.idCapa, i)
37.             self.ShowAverageGray()
38.
39. if __name__ == '__main__':
40.     demo = Tucam()
41.     demo.OpenCamera(0)
42.     if demo.TUCAMOPEN.hIdxTUCam != 0:
43.         demo.SetResolution()
44.         demo.CloseCamera()
```

```
45. demo.UnInitApi()
```

4.2.3 属性获取和设置

4.2.3.1 调用顺序

1) 获取属性

TUCAM_Prop_GetAttr 函数，通过设置属性 ID 获取属性属性，其中包括最大值、最小值、默认值。

2) 查询、设置属性值

通过 TUCAM_Prop_GetValue 和 TUCAM_Prop_SetValue 函数来设置和获取属性值。所有的相机属性值都被处理成一个双精度浮点型，某些属性具有多个属性值。设置和获取属性通常在相机进行捕获之前或者之后已经完成。如果设置函数在数据捕获时被调用，在某些情况下可能会返回错误的代码 TUCAMRET 。[接口参考 5.3.4](#)

3) 获取性能值文本

TUCAM_Prop_GetValueText 函数获取值文本，用于描述每个属性值具体含义。

4.2.3.2 属性索引

每个属性都有唯一的 ID，参考 [TUCAM_IDPROP](#)

注意：如果相机不支持该属性 ID，将返回错误代码 TUCAMRET_NOT_SUPPORT 或 TUCAMRET_INVALID_IDPROP。

4.2.3.3 示例代码

参考以下代码或 samples code 目录 25_set_exposure.py 工程：

```
1. class Tucam():
2.     def __init__(self):
3.         self.Path = './'
4.         self.TUCAMINIT = TUCAM_INIT(0, self.Path.encode('utf-8'))
```

```
5.         self.TUCAMOPEN = TUCAM_OPEN(0, 0)
6.         TUCAM_Api_Init(pointer(self.TUCAMINIT))
7.         print(self.TUCAMINIT.uiCamCount)
8.         print(self.TUCAMINIT.pstrConfigPath)
9.         print('Connect %d camera' %self.TUCAMINIT.uiCamCount)
10.
11.     def OpenCamera(self, ldx):
12.         if ldx >= self.TUCAMINIT.uiCamCount:
13.             return
14.         self.TUCAMOPEN = TUCAM_OPEN(ldx, 0)
15.         TUCAM_Dev_Open(pointer(self.TUCAMOPEN))
16.         if 0 == self.TUCAMOPEN.hIdxTUCam:
17.             print('Open the camera failure!')
18.             return
19.         else:
20.             print('Open the camera success!')
21.
22.     def CloseCamera(self):
23.         if 0 != self.TUCAMOPEN.hIdxTUCam:
24.             TUCAM_Dev_Close(self.TUCAMOPEN.hIdxTUCam)
25.             print('Close the camera success')
26.
27.     def UnInitApi(self):
28.         TUCAM_Api_Uninit()
29.
30.     # ch:设置曝光时间 | en:Set exposure time
31.     def SetExposure(self, value):
32.         TUCAM_Capa_SetValue(self.TUCAMOPEN.hIdxTUCam,
33.                               TUCAM_IDCAPA.TUIDC_ATEXPOSURE.value, 0)
34.         TUCAM_Prop_SetValue(self.TUCAMOPEN.hIdxTUCam,
35.                               TUCAM_IDPROP.TUIDP_EXPOSURETM.value, value, 0);
36.         print("Set exposure:", value)
37.
38. if __name__ == '__main__':
39.     demo = Tucam()
40.     demo.OpenCamera(0)
41.     if demo.TUCAMOPEN.hIdxTUCam != 0:
42.         # ch:设置 10ms 曝光时间 | en:Set 10ms exposure time
43.         demo.SetExposure(10)
44.         # ch:设置 20ms 曝光时间 | en:Set 20ms exposure time
45.         demo.SetExposure(20)
46.         # ch:设置 30ms 曝光时间 | en:Set 30ms exposure time
```

```
45.         demo.SetExposure(30)
46.         demo.CloseCamera()
47.     demo.UnInitApi()
```

4.2.4 内存管理

4.2.4.1 调用顺序

1) 内存的分配

内存的分配 `TUCAM_Buf_Alloc` 必须在 `TUCAM_Cap_Start` 数据开始捕获之前调用，且切换不同的分辨率都必须重新进行内存分配。

2) 数据的获取

必须在 `TUCAM_Cap_Start` 数据开始捕获之后调用 `TUCAM_Buf_WaitForFrame`

等待数据捕获的完成，并且可以通过 `TUCAM_Buf_CopyFrame` 拷贝当前帧的不同格式数据，一旦再次调用 `TUCAM_Buf_WaitForFrame`, `TUCAM_Buf_CopyFrame` 拷贝也会更新为下一帧数据。

3) 结束等待

如有进行数据等待和数据拷贝的调用，在停止数据捕获之前调用 `TUCAM_Buf_AbortWait` 结束数据等待，之后再调用 `TUCAM_Cap_Stop` 停止数据捕获。

4) 内存释放

内存的释放 `TUCAM_Buf_Release` 必须在 `TUCAM_Cap_Stop` 停止数据捕获之后调用。

4.2.4.2 帧结构体

用于描述一帧图像数据的结构体。参考 [TUCAM_FRAME](#)。变量 `pBuffer` 是指向帧数据指针，帧数据包括帧头部数据、帧图像数据两部分。将帧数据偏移 `usHeader` 字节获取到帧图像数据。

4.2.4.3 示例代码

参考以下代码或 samples code 目录 05_wait_frame.py 工程：

```
1. class Tucam():
2.     def __init__(self):
3.         self.Path = './'
4.         self.TUCAMINIT = TUCAM_INIT(0, self.Path.encode('utf-8'))
5.         self.TUCAMOPEN = TUCAM_OPEN(0, 0)
6.         # ch:初始化相机枚举相机个数 | en:Initializing Cameras Enumerates the number of
cameras
7.         TUCAM_Api_Init(pointer(self.TUCAMINIT))
8.         print(self.TUCAMINIT.uiCamCount)
9.         print(self.TUCAMINIT.pstrConfigPath)
10.        print('Connect %d camera' %self.TUCAMINIT.uiCamCount)
11.
12.        def OpenCamera(self, ldx):
13.
14.            if ldx >= self.TUCAMINIT.uiCamCount:
15.                return
16.
17.            self.TUCAMOPEN = TUCAM_OPEN(ldx, 0)
18.
19.            # ch:打开相机 ldx | en:Open camera ldx
20.            TUCAM_Dev_Open(pointer(self.TUCAMOPEN))
21.
22.            if 0 == self.TUCAMOPEN.hIdxTUCam:
23.                print('Open the camera failure!')
24.                return
25.            else:
26.                print('Open the camera success!')
27.
28.        def CloseCamera(self):
29.            # ch:关闭相机 | en:Close camera
30.            if 0 != self.TUCAMOPEN.hIdxTUCam:
31.                TUCAM_Dev_Close(self.TUCAMOPEN.hIdxTUCam)
32.                print('Close the camera success')
33.
34.        def UnInitApi(self):
35.            # ch:反初始化相机 | en:Uninitial Cameras
36.            TUCAM_Api_Uninit()
```

```
37.
38.     def WaitForImageData(self):
39.         m_frame = TUCAM_FRAME()
40.         m_format = TUIMG_FORMATS
41.         m_frformat = TUFRM_FORMATS
42.         m_capmode = TUCAM_CAPTURE_MODES
43.
44.         m_frame.pBuffer = 0
45.         m_frame.ucFormatGet = m_frformat.TUFRM_FMT_USUAL.value
46.         m_frame.uiRsdSize = 1
47.
48.         # ch:分配内存 | en:Alloc buffer
49.         TUCAM_Buf_Alloc(self.TUCAMOPEN.hIdxTUCam, pointer(m_frame))
50.         # ch:开始采集 | en:Start capture
51.         TUCAM_Cap_Start(self.TUCAMOPEN.hIdxTUCam,
m_capmode.TUCCM_SEQUENCE.value)
52.
53.         nTimes = 10
54.         for i in range(nTimes):
55.             try:
56.                 # ch:等待数据帧 | en:Wait for frames
57.                 result = TUCAM_Buf_WaitForFrame(self.TUCAMOPEN.hIdxTUCam,
pointer(m_frame), 1000)
58.                 print(
59.                     "Grab the frame success, index number is %#d, width:%d, height:%#d,
channel:%#d, elembytes:%#d, image size:%#d"%(i, m_frame.usWidth, m_frame.usHeight,
m_frame.ucChannels,
60.                     m_frame.ucElemBytes, m_frame.uImgSize)
61.                 )
62.             except Exception:
63.                 print('Grab the frame failure, index number is %#d', i)
64.                 continue
65.
66.         # ch:跳出等待数据帧 | en:Abort wait for frames
67.         TUCAM_Buf_AbortWait(self.TUCAMOPEN.hIdxTUCam)
68.         # ch:停止采集 | en:Stop capture
69.         TUCAM_Cap_Stop(self.TUCAMOPEN.hIdxTUCam)
70.         # ch:释放内存 | en:Release buffer
71.         TUCAM_Buf_Release(self.TUCAMOPEN.hIdxTUCam)
72.
73. if __name__ == '__main__':
74.     demo = Tucam()
```

```
75.     demo.OpenCamera(0)
76.     if demo.TUCAMOPEN.hIdxTUCam != 0:
77.         demo.WaitForImageData()
78.         demo.CloseCamera()
79.     demo.UnInitApi()
```

4.2.5 捕获控制

4.2.5.1 调用顺序

设置 ROI 属性 TUCAM_Cap_SetROI 和触发属性 TUCAM_Cap_SetTrigger，需要在开始捕获数据之前调用，如果在数据捕获时调用可能返回 TUCAMRET 错误代码。

4.2.5.2 捕获模式索引

参考 [TUCAM_CAPTURE_MODES](#)

4.2.5.3 示例代码

参考以下代码或 samples code 目录 06_roi_mode.py 和 11_trigger_hardware.py 工程：

```
1.  // 设置 ROI
2.  def SetROI(self):
3.      roi = TUCAM_ROI_ATTR()
4.      roi.bEnable = 1
5.      roi.nHOffset = 0
6.      roi.nVOffset = 0
7.      roi.nWidth = 320
8.      roi.nHeight = 240
9.      try:
10.         # ch:设置相机感兴趣区域 | en:Set ROI
11.         TUCAM_Cap_SetROI(self.TUCAMOPEN.hIdxTUCam, roi)
12.         print('Set ROI state success, HOffset: %#d, VOffset: %#d, Width: %#d,
13.             Height: %#d'%(roi.nHOffset,
14.                 roi.nVOffset, roi.nWidth, roi.nHeight))
15.     except Exception:
16.         print('Set ROI state failure, HOffset: %#d, VOffset: %#d, Width: %#d,
```

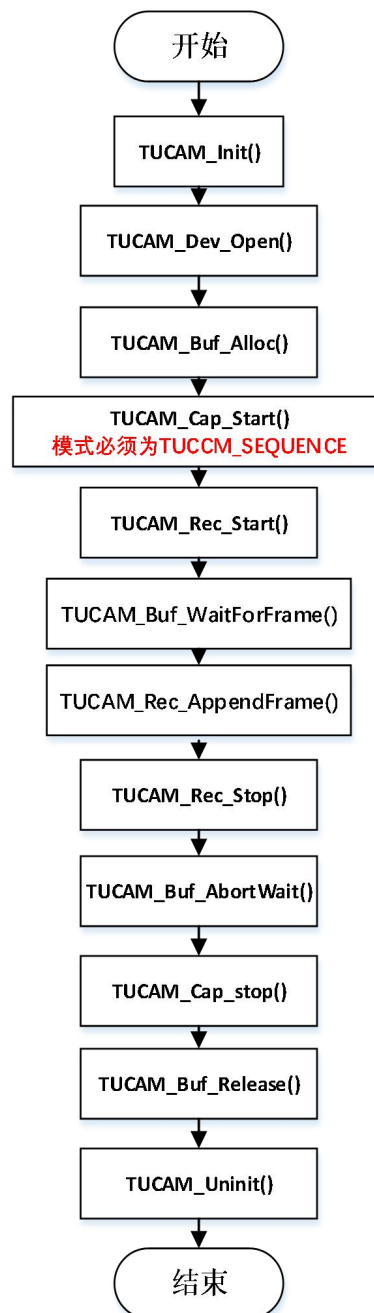
```
Height: %#d' % (roi.nHOffset,
16.             roi.nVOffset, roi.nWidth, roi.nHeight))
17. // 设置触发模式
18. def DoHardwareTrigger(self):
19.     m_tgr = TUCAM_TRIGGER_ATTR()
20.     m_frame = TUCAM_FRAME()
21.     m_format = TUIMG_FORMATS
22.     m_frformat = TUFRM_FORMATS
23.     m_capmode = TUCAM_CAPTURE_MODES
24.
25.     # ch:设置相机标准触发模式 | en:Set standard trigger mode
26.     TUCAM_Cap_GetTrigger(self.TUCAMOPEN.hIdxTUCam, pointer(m_tgr))
27.     m_tgr.nTgrMode = m_capmode.TUCCM_TRIGGER_STANDARD.value
28.     m_tgr.nBufFrames = 1
29.     TUCAM_Cap_SetTrigger(self.TUCAMOPEN.hIdxTUCam, m_tgr)
30.
31.     m_frame.pBuffer = 0
32.     m_frame.ucFormatGet = m_frformat.TUFRM_FMT_USUAL.value
33.     m_frame.uiRsdSize = 1
34.
35.     TUCAM_Buf_Alloc(self.TUCAMOPEN.hIdxTUCam, pointer(m_frame))
36.     TUCAM_Cap_Start(self.TUCAMOPEN.hIdxTUCam, m_tgr.nTgrMode)
37.
```

4.2.6 文件管理

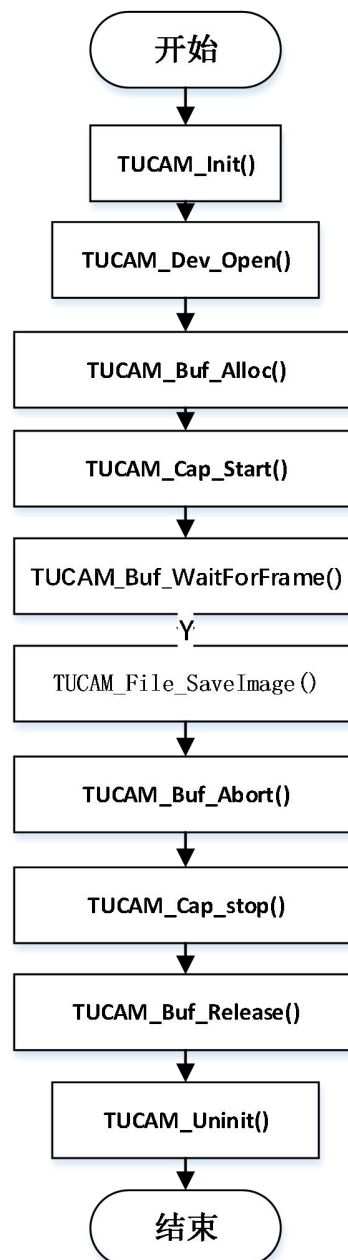
4.2.6.1 调用顺序

1) 视频流保存流程

录像开始 TUCAM_Rec_Start 需要在 TUCAM_Cap_Start 开始捕获数据之后调用，并且设置的捕获方式必须是 TUCCM_SEQUENCE 模式。录像过程中通过调用 TUCAM_Rec_AppendFrame 要将图像数据以追加的方式写入文件中，录像结束调用 TUCAM_Rec_Stop 来结束录像过程，再调用 TUCAM_CAP_STOP 结束采集。



2) 图片保存流程:



4.2.6.2 文件结构体

文件保存结构体参考 [TUCAM_File_Save](#)

录像保存结构体参考 [TUCAM_Rec_Save](#)

4.2.6.3 示例代码

参考以下代码或 samples code 目录 07_save_image.py/08_save_video.py 工程：

```
1. // 保存图片文件
2. def SaveImageData(self):
3.     m_fs = TUCAM_FILE_SAVE()
4.     m_frame = TUCAM_FRAME()
5.     m_format = TUIMG_FORMATS
6.     m_frformat = TUFRM_FORMATS
7.     m_capmode = TUCAM_CAPTURE_MODES
8.
9.     m_frame.pBuffer = 0
10.    m_frame.ucFormatGet = m_frformat.TUFRM_FMT_USUAL.value
11.    m_frame.uiRsdSize = 1
12.
13.    m_fs.nSaveFmt = m_format.TUFMT_TIF.value
14.
15.    TUCAM_Buf_Alloc(self.TUCAMOPEN.hIdxTUCam, pointer(m_frame))
16.    TUCAM_Cap_Start(self.TUCAMOPEN.hIdxTUCam,
17.        m_capmode.TUCCM_SEQUENCE.value)
18.
19.    nTimes = 10
20.    for i in range(nTimes):
21.        try:
22.            result = TUCAM_Buf_WaitForFrame(self.TUCAMOPEN.hIdxTUCam,
23.                pointer(m_frame), 1000)
24.            ImgName = './Image_' + str(i)
25.            m_fs.pFrame = pointer(m_frame)
26.            m_fs.pstrSavePath = ImgName.encode('utf-8')
27.            # ch:保存数据帧到硬盘 | en:Save image to disk
28.            TUCAM_File_SaveImage(self.TUCAMOPEN.hIdxTUCam, m_fs)
29.            print('Save the image data success, the path is %s'%ImgName)
30.        except Exception:
31.            print('Grab the frame failure, index number is %d'%i)
32.            continue
33.
34.    TUCAM_Buf_AbortWait(self.TUCAMOPEN.hIdxTUCam)
35.    TUCAM_Cap_Stop(self.TUCAMOPEN.hIdxTUCam)
36.    TUCAM_Buf_Release(self.TUCAMOPEN.hIdxTUCam)
37. // 保存录像文件
38. def SaveVideo(self):
39.     m_rs = TUCAM_REC_SAVE()
40.     m_frame = TUCAM_FRAME()
41.     m_format = TUIMG_FORMATS
42.     m_frformat = TUFRM_FORMATS
```

```
41.         m_capmode = TUCAM_CAPTURE_MODES
42.
43.         m_frame.pBuffer = 0
44.         m_frame.ucFormatGet = m_frformat.TUFRM_FMT_USUAL.value
45.         m_frame.uiRsdSize = 1
46.
47.         TUCAM_Buf_Alloc(self.TUCAMOPEN.hIdxTUCam, pointer(m_frame))
48.         TUCAM_Cap_Start(self.TUCAMOPEN.hIdxTUCam,
49.             m_capmode.TUCCM_SEQUENCE.value)
49.         # ch:录像存储到硬盘 | en:Save video to disk
50.         strPath = './example.avi'
51.         m_rs.fFps = 25.0
52.         m_rs.nCodec = 0
53.         m_rs.pstrSavePath = strPath.encode('utf-8')
54.         TUCAM_Rec_Start(self.TUCAMOPEN.hIdxTUCam, m_rs)
55.         print('Start video success')
56.         nTimes = 50
57.         for i in range(nTimes):
58.             try:
59.                 result = TUCAM_Buf_WaitForFrame(self.TUCAMOPEN.hIdxTUCam,
60.                     pointer(m_frame), 1000)
61.                 TUCAM_Rec_AppendFrame(self.TUCAMOPEN.hIdxTUCam,
62.                     pointer(m_frame))
63.                 print('Save the frames of video success, index number is %#d'%i)
64.             except Exception:
65.                 print('Grab the frame failure, index number is %#d'%i)
66.                 continue
67.
68.         TUCAM_Rec_Stop(self.TUCAMOPEN.hIdxTUCam)
69.         TUCAM_Buf_AbortWait(self.TUCAMOPEN.hIdxTUCam)
70.         TUCAM_Cap_Stop(self.TUCAMOPEN.hIdxTUCam)
71.         TUCAM_Buf_Release(self.TUCAMOPEN.hIdxTUCam)
```

4.2.7 扩展控制

4.2.7.1 调用顺序

读写寄存器 TUCAM_Reg_Read / TUCAM_Reg_Write 的调用必须在

TUCAM_Dev_Open 打开相机后调用, 如果 TUCAM_Dev_Close 关闭相机后将无法读写寄存器, 必须重新打开相机。

4.2.7.2 寄存器结构体

参考 [TUCAM_REG_RW](#) 结构体

4.2.7.3 示例代码

```
1. // 读取寄存器数据
2. cSN = (c_char * 64)()
3. pSN = cast(cSN, c_char_p)
4. TUCAMREGRW = TUCAM_REG_RW(1, pSN, 64)
5. TUCAM_Reg_Read(TUCAMOPEN.hIdxTUCam, TUCAMREGRW)
6. print(string_at(pSN))
```

4.2.8 其它

4.2.8.1 制冷温度设置

```
1. // 获取温度
2. value= 0
3. TUCAM_Prop_GetValue(TUCAMOPEN.hIdxTUCam,
    TUCAM_IDPROP.TUIDP_TEMPERATURE.value, pointer(value), 0)
4. // 设置温度
5. value= 40 // 设置-10℃ (40 - 50 = -10)
6. TUCAM_Prop_SetValue(TUCAMOPEN.hIdxTUCam,
    TUCAM_IDPROP.TUIDP_TEMPERATURE.value, c_double(value), 0)
```

4.2.8.2 自动 / 一次曝光 / 自动曝光模式

```
1. // 相机设置自动曝光
2. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,
    TUCAM_IDCAPA.TUIDC_ATEXPOSURE.value, 1);
3.
4. // 相机设置一次曝光
5. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,
```

```
TUCAM_IDCAPA.TUIDC_ATEXPOSURE.value, 2);
6. // 一直等待获取
7. value= 0;
8. TUCAM_Capa_GetValue(TUCAMOPEN.hIdxTUCam,
    TUCAM_IDCAPA.TUIDC_ATEXPOSURE.value, pointer(value));
9. // 一次曝光结束
10. If 0 == value:
11.
12. // 默认为居右曝光模式（仅 Mlchrome 支持），此时不支持自动曝光目标值
    （TUIDP_BRIGHTNESS）
13. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,TUCAM_IDCAPA.TUIDC_ATEXPOSU
    RE_MODE.value, 0);
14.
15. // 设置居中曝光模式，此时支持自动曝光目标值的设置
16. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,TUCAM_IDCAPA.TUIDC_ATEXPOSU
    RE_MODE.value, 3);
17. TUCAM_Prop_SetValue(TUCAMOPEN.hIdxTUCam,
    TUCAM_IDPROP.TUIDP_BRIGHTNESS.value, 128);
18.
```

4.2.8.3 自动色阶调用

```
1. // 自动设置模式(0 关闭,1 自动左色阶,2 自动右色阶,3 自动左右色阶)
2. value = 0;
3. If 0 == value:
4. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,TUCAM_IDCAPA.TUIDC_HISTC.value
    , 0); // 关闭直方图统计
5. else:
6. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,TUCAM_IDCAPA.TUIDC_HISTC.value
    , 1); // 开启直方图统计
7.
8. // 设置自动色阶模式
9. TUCAM_Capa_SetValue(TUCAMOPEN.hIdxTUCam,TUCAM_IDCAPA.TUIDC_ATLEVELS.
    value, value);
10.
```

4.2.8.4 计算平均灰度值

```
1.  def CalculateMonoAverageGray(self, m_frame):
2.      buf = create_string_buffer(m_frame.uImageSize)
3.      pointer_data = c_void_p(m_frame.pBuffer + m_frame.usHeader)
4.      memmove(buf, pointer_data, m_frame.uImageSize)
5.      sum = c_double()
6.      avg = c_double()
7.      sum = 0
8.      size = m_frame.usWidth * m_frame.usHeight
9.      buffers = bytes(buf)
10.     for i in range(size):
11.         if 1 == m_frame.ucElemBytes:
12.             value = buffers[i]
13.             sum += value
14.         else:
15.             value = buffers[i*2] + buffers[i*2+1] * 256
16.             sum += value
17.     avg = sum / size
18.     print("Grab the frame average is:", avg)
19.
20.  def CalculateColorAverageGray(self, m_frame):
21.      buf = create_string_buffer(m_frame.uImageSize)
22.      pointer_data = c_void_p(m_frame.pBuffer + m_frame.usHeader)
23.      memmove(buf, pointer_data, m_frame.uImageSize)
24.      sum = c_double()
25.      avg = c_double()
26.      sum = 0
27.      size = m_frame.usWidth * m_frame.usHeight
28.      buffers = bytes(buf)
29.      for i in range(size):
30.          if 1 == m_frame.ucElemBytes:
31.              value = buffers[i*3] * 0.114 + buffers[i*3+1] * 0.587 + buffers[i*3+2] * 0.299
32.              sum += value
33.          else:
34.              value = (buffers[i*6] + buffers[i*6+1] * 256) * 0.114 + (buffers[i*6+2] +
35. buffers[i*6+3] * 256) * 0.587 + (buffers[i*6+4] + buffers[i*6+5] * 256) * 0.299
36.              sum += value
37.      avg = sum / size
38.      print("Grab the frame average is:", avg)
39.      // 计算平均灰度值
40.  def CalculateAverageGray(self, m_frame):
```

```

40.         if 1 == m_frame.ucChannels:
41.             self.CalculateMonoAverageGray(m_frame)
42.         else:
43.             self.CalculateColorAverageGray(m_frame)
44.

```

5. 参考

5.1 类型和常量

5.1.1 TUCAMRET 错误代码

错误代码常量 TUCAMRET_ 为前缀，按照类型分为初始化错误、状态错误、等待错误、调用错误、相机或接口错误。

名称	错误代码	描述
TUCAMRET_SUCCESS	0x00000001	没有错误，成功代码，应用程序应检查值为正
TUCAMRET_RECEIVE_FINISH	0x00000002	没有错误，厂商收到帧信息
TUCAMRET_EXTERNAL_TRIGGER	0x00000003	没有错误，接收到外部触发信号
TUCAMRET_FAILURE	0x80000000	调用 API 接口失败
初始化错误		
TUCAMRET_NO_MEMORY	0x80000101	没有足够的内存
TUCAMRET_NO_RESOURCE	0x80000102	没有足够的资源（不包括内存）
TUCAMRET_NO_MODULE	0x80000103	没有支持的子模块
TUCAMRET_NO_DRIVER	0x80000104	没有支持的驱动
TUCAMRET_NO_CAMERA	0x80000105	没有连接的相机
TUCAMRET_NO_GRABBER	0x80000106	没有取图
TUCAMRET_NO_PROPERTY	0x80000107	没有代替的属性 ID
TUCAMRET_FAILOPEN_CAMERA	0x80000110	打开相机失败
TUCAMRET_FAILOPEN_BULKIN	0x80000111	打开批传输输入端点失败（USB 接口）
TUCAMRET_FAILOPEN_BULKOUT	0x80000112	打开批传输输出端点失败（USB 接口）
TUCAMRET_FAILOPEN_CONTROL	0x80000113	打开控制端点失败
TUCAMRET_FAILCLOSE_CAMERA	0x80000114	关闭相机失败
TUCAMRET_FAILOPEN_FILE	0x80000115	打开文件失败
TUCAMRET_FAILOPEN_CODEC	0x80000116	打开编码器失败
TUCAMRET_FAILOPEN_CONTEXT	0x80000117	打开上下文失败

状态错误		
TUCAMRET_INIT	0x80000201	API 需要初始化状态
TUCAMRET_BUSY	0x80000202	API 处于繁忙状态
TUCAMRET_NOT_INIT	0x80000203	API 未初始化
TUCAMRET_EXCLUDED	0x80000204	一些资源被独占使用
TUCAMRET_NOT_BUSY	0x80000205	API 未处于繁忙状态
TUCAMRET_NOT_READY	0x80000206	API 未处于就绪状态
等待错误		
TUCAMRET_ABORT	0x80000207	终止处理
TUCAMRET_TIMEOUT	0x80000208	超时
TUCAMRET_LOSTFRAME	0x80000209	帧丢失
TUCAMRET_MISSFRAME	0x8000020A	帧丢失但是是底层驱动的问题
TUCAMRET_USB_STATUS_ERROR	0x8000020B	USB 状态错误
调用错误		
TUCAMRET_INVALID_CAMERA	0x80000301	无效相机
TUCAMRET_INVALID_HANDLE	0x80000302	无效相机句柄
TUCAMRET_INVALID_OPTION	0x80000303	无效配置的值
TUCAMRET_INVALID_IDPROP	0x80000304	无效属性 ID
TUCAMRET_INVALID_IDCAPA	0x80000305	无效性能 ID
TUCAMRET_INVALID_IDPARAM	0x80000306	无效参数 ID
TUCAMRET_INVALID_PARAM	0x80000307	无效参数
TUCAMRET_INVALID_FRAMEIDX	0x80000308	无效帧序列号
TUCAMRET_INVALID_VALUE	0x80000309	无效值
TUCAMRET_INVALID_EQUAL	0x8000030A	值相等，参数无效
TUCAMRET_INVALID_CHANNEL	0x8000030B	属性 ID 指定通道，但通道无效
TUCAMRET_INVALID_SUBARRAY	0x8000030C	子数组的值是无效的
TUCAMRET_INVALID_VIEW	0x8000030D	无效的显示窗口句柄
TUCAMRET_INVALID_PATH	0x8000030E	无效的文件路径
TUCAMRET_INVALID_IDVPROP	0x8000030F	无效的厂商属性 ID
TUCAMRET_NO_VALUETEXT	0x80000310	属性没有值的文本
TUCAMRET_OUT_OF_RANGE	0x80000311	值超出范围
TUCAMRET_NOT_SUPPORT	0x80000312	相机不支持性能或属性
TUCAMRET_NOT_WRITABLE	0x80000313	属性不可写
TUCAMRET_NOT_READABLE	0x80000314	属性不可读
TUCAMRET_WRONG_HANDSHAKE	0x80000410	错误发生在获取错误代码时
TUCAMRET_NEWAPI_REQUIRED	0x80000411	旧 API 不支持，只有新 API 支持
TUCAMRET_ACCESSDENY	0x80000412	访问拒绝（可能是没有足够权限）
TUCAMRET_NO_CORRECTIONDATA	0x80000501	没有彩色校点数据
TUCAMRET_INVALID_PRFSETS	0x80000601	配置文件设置名称无效

TUCAMRET_INVALID_IDPPROP	0x80000602	无效的属性 ID
TUCAMRET_DECODE_FAILURE	0x80000701	解码失败
TUCAMRET_COPYDATA_FAILURE	0x80000702	拷贝数据失败
TUCAMRET_ENCODE_FAILURE	0x80000703	编码失败
TUCAMRET_WRITE_FAILURE	0x80000704	写入失败
相机或总线错误		
TUCAMRET_FAIL_READ_CAMERA	0x83001001	从相机读取失败
TUCAMRET_FAIL_WRITE_CAMERA	0x83001002	写入相机失败
TUCAMRET_OPTICS_UNPLUGGED	0x83001003	光学部件已拆下，请检查

5.1.2 TUCAM_IDINFO 产品信息代码

名称	代码	描述
TUIDI_BUS	0x00	USB 接口类型。0x200、0x210 对应 USB2.0。
TUIDI_VENDOR	0x01	厂商 ID
TUIDI_PRODUCT	0x02	产品 ID
TUIDI_VERSION_API	0x03	当前使用 TUCAM-API 接口版本
TUIDI_VERSION_FRMW	0x04	当前相机固件版本号
TUIDI_VERSION_FPGA	0x06	当前相机 FPGA 版本号
TUIDI_VERSION_DRIVER	0x07	当前相机驱动版本号
TUIDI_TRANSFER_RATE	0x08	传输速率
TUIDI_CAMERA_MODEL	0x09	当前相机型号，字符串类型（例如 Dhyana 400BSI V3）
TUIDI_CURRENT_WIDTH	0x0A	相机图像数据宽度（必须使用 TUCAM_Dev_GetInfoEx, 并在 TUCAM_Buf_Alloc 后调用）
TUIDI_CURRENT_HEIGHT	0x0B	相机图像数据高度（必须使用 TUCAM_Dev_GetInfoEx, 并在 TUCAM_Buf_Alloc 后调用）
TUIDI_CAMERA_CHANNELS	0x0C	相机图像数据通道数。彩色相机为 3，黑白相机为 1。
TUIDI_BCDDEVICE	0x0D	BCD 码
TUIDI_TEMPALARMFLAG	0x0E	温度预警标志
TUIDI_UTCTIME	0x0F	获取 UTC（世界统一时间、世界标准时间）
TUIDI_LONGITUDE_LATITUDE	0x10	获取经纬度
TUIDI_WORKING_TIME	0x11	获取工作时间
TUIDI_FAN_SPEED	0x12	获取风扇速度
TUIDI_FPGA_TEMPERATURE	0x13	获取 FPGA 温度
TUIDI_PCBA_TEMPERATURE	0x14	获取 PCBA 温度
TUIDI_ENV_TEMPERATURE	0x15	获取环境温度

TUIDI_DEVICE_ADDRESS	0x16	USB 设备地址
TUIDI_USB_PORT_ID	0x17	USB 设备端口号
TUIDI_ENDINFO	0x18	产品信息 ID 结束位

5.1.3 TUCAM_IDCAPA 性能代码

名称	代码	描述	读/写
TUIDC_RESOLUTION	0x00	分辨率	R/W
TUIDC_PIXELCLOCK	0x01	像素时钟	R/W
TUIDC_BITOFDEPTH	0x02	数据位宽 (8bit, 16bit)	R/W
TUIDC_ATEXPOSURE	0x03	自动曝光 (0: 关闭 1: 打开)	R/W
TUIDC_HORIZONTAL	0x04	水平镜像 (0: 关闭 1: 打开)	R/W
TUIDC_VERTICAL	0x05	垂直镜像 (0: 关闭 1: 打开)	R/W
TUIDC_ATWBALANCE	0x06	自动白平衡 (仅彩色相机支持) (0: 关闭 1: 打开)	R/W
TUIDC_FAN_GEAR	0x07	风扇档位 (仅制冷相机支持)	R/W
TUIDC_ATLEVELS	0x08	自动色阶[0, 3] 0: 关闭 (1: 左色阶 2: 右色阶 3: 左右色阶)	R/W
TUIDC_SHIFT	0x09	显示移位[16bit 有效], 选取其中连续 8 位(15~8, 14~7, 13~6, 12~5, 11~4, 10~3, 9~2, 8~1, 7~0)	R/W
TUIDC_HISTC	0x0A	直方图统计使能 (0: 关闭 1: 打开)	R/W
TUIDC_CHANNELS	0x0B	当前通道索引 (参考 TUCHN_SELECT, 仅彩色相机)	R/W
TUIDC_ENHANCE	0x0C	图像增强使能	R/W
TUIDC_DFTCORRECTION	0x0D	坏点校正 (某几款相机支持) (0: 不校正 1: 计算 2: 校正)	R/W
TUIDC_ENABLEDENOISE	0x0E	降噪使能, 受降噪等级影响 (参考 TUIDP_NOISELEVEL)	R/W
TUIDC_FLTCORRECTION	0x0F	平场校正 0: 关闭 (1: 抓取帧 2: 计算 3: 校正)	R/W
TUIDC_RESTARTLONGTM	0x10	重启长时间曝光 (仅 CCD 相机支持)	R/W
TUIDC_DATAFORMAT	0x11	数据格式 (某款相机支持) (仅支持 0: YUV 1: RAW 切换)	R/W
TUIDC_DRCORRECTION	0x12	动态范围校正	R/W
TUIDC_VERCORRECTION	0x13	垂直镜像校正 (校正数据垂直显示, 在 Windows 系统中默认值为 1)	R/W
TUIDC_MONOCHROME	0x14	单色使能	R/W
TUIDC_BLACKBALANCE	0x15	黑平衡使能	R/W
TUIDC_IMGMODESELECT	0x16	图像模式选择 (CMS 模式使能)	R/W

TUIDC_CAM_MULTIPLE	0x17	多相机设置 (设置同时捕获数据的相机个数)	RO
TUIDC_ENABLEPOWEEFREQUENCY	0x18	消频闪使能 (0: 50hz 1:60hz)	R/W
TUIDC_ROTATE_R90	0x19	顺时针转 90 度使能	R/W
TUIDC_ROTATE_L90	0x1A	逆时针转 90 度使能	R/W
TUIDC_NEGATIVE	0x1B	负片使能	R/W
TUIDC_HDR	0x1C	HDR 使能	R/W
TUIDC_ENABLEIMGPRO	0x1D	图像处理使能	R/W
TUIDC_ENABLELED	0x1E	相机 LED 灯使能	R/W
TUIDC_TUIDC_ENABLETIMESTAMP	0x1F	时间戳使能	R/W
TUIDC_ENABLEBLACKLEVEL	0x20	黑电平	R/W
TUIDC_ATFOCUS	0x21	自动对焦使能 (0: 手动 1: 自动对焦 2: 一次对焦)	R/W
TUIDC_ATFOCUS_STATUS	0x22	自动对焦状态 (参考 TUFOCUS_STATUS)	R/W
TUIDC_PGAGAIN	0x23	传感器增益(针对某些相机仅包含传感器 高增益或传感器低增益其中一种)	R/W
TUIDC_ATEXPOSURE_MODE	0x24	自动曝光模式 (0: 居中曝光 3: 居右曝光, 支持目标 值)	R/W
TUIDC_BINNING_SUM	0x25	软件 Binning, 像素求和模式	R/W
TUIDC_BINNING_AVG	0x26	软件 Binning, 像素求平均模式	R/W
TUIDC_FOCUS_C_MOUNT	0x27	C-Mount 聚焦模式 (0: 普通模式 1: C-Mount 模式)	R/W
TUIDC_ENABLEPI	0x28	PI (加热膜) 使能	R/W
TUIDC_ATEXPOSURE_STATUS	0x29	自动曝光状态 (0: 正在执行 1: 完成)	RO
TUIDC_ATWBALANCE_STATUS	0x2A	自动白平衡 (0: 正在执行 1: 完成)	RO
TUIDC_TESTIMGMODE	0x2B	测试图片模式 (多种测试图, 与拍摄场景无关)	R/W
TUIDC_SENSORRESET	0x2C	传感器重置 (设置一次)	WO
TUIDC_PGAHIGH	0x2D	传感器高增益	R/W
TUIDC_PGALOW	0x2E	传感器低增益	R/W
TUIDC_PIXCLK1_EN	0x2F	像素时钟 1 使能	R/W
TUIDC_PIXCLK2_EN	0x30	像素时钟 2 使能	R/W
TUIDC_ATLEVELGEAR	0x31	自动色阶档位	R/W
TUIDC_ENABLEDSNU	0x32	暗信号非均一性使能	R/W
TUIDC_ENABLEOVERLAP	0x33	启用曝光时间重叠模式	R/W
TUIDC_CAMSTATE	0x34	相机状态	R/W
TUIDC_ENABLETRIOUT	0x35	触发输出使能	R/W
TUIDC_ROLLINGSCANMODE	0x36	卷帘扫描模式使能	R/W
TUIDC_ROLLINGSCANLTD	0x37	卷帘扫描行延迟时间	R/W

TUIDC_ROLLINGSCANSLIT	0x38	卷帘扫描间隔高度	R/W
TUIDC_ROLLINGSCANDIR	0x39	卷帘扫描方向 (0: 向下 1: 向上 2: 自下而上循环)	R/W
TUIDC_ROLLINGSCANRESET	0x3A	卷帘扫描方向重置	R/W
TUIDC_ENABLETEC	0x3B	TEC (制冷开关) 使能	R/W
TUIDC_ENABLEBLC	0x3C	背光补偿使能	R/W
TUIDC_ENABLETHROUGHFOG	0x3D	电子透雾使能 (有等级) 图像处理	R/W
TUIDC_ENABLEGAMMA	0x3E	伽马使能	R/W
TUIDC_ENABLEFILTER	0x3F	滤波使能	R/W
TUIDC_ENABLEHLC	0x40	强光抑制使能	R/W
TUIDC_CAMPARASAVE	0x41	相机参数保存	R/W
TUIDC_CAMPARALOAD	0x42	相机参数读取	R/W
TUIDC_ENABLEISP	0x43	相机 ISP 算法使能	R/W
TUIDC_ENDCAPABILITY	0x44	相机性能 ID 结束位	R/W

5.1.4 TUCAM_IDPROP 属性代码

名称	代码	描述	读/写
TUIDP_GLOBALGAIN	0x00	全局增益	R/W
TUIDP_EXPOSURETM	0x01	曝光时间 ms	R/W
TUIDP_BRIGHTNESS	0x02	自动曝光目标值 (居右自动曝光状态有效)	R/W
TUIDP_BLACKLEVEL	0x03	黑电平	R/W
TUIDP_TEMPERATURE	0x04	相机 (Sensor) 温度	R/W
TUIDP_SHARPNESS	0x05	锐化等级	R/W
TUIDP_NOISELEVEL	0x06	降噪等级	R/W
TUIDP_HDR_KVALUE	0x07	HDR 的 K 值 (sCMOS 相机支持)	R/W
图片处理属性			
TUIDP_GAMMA	0x08	伽玛	R/W
TUIDP_CONTRAST	0x09	对比度	R/W
TUIDP_LFTLEVELS	0x0A	左色阶	R/W
TUIDP_RGTLEVELS	0x0B	右色阶	R/W
TUIDP_CHNLGAIN	0x0C	通道增益(彩色相机支持)	R/W
TUIDP_SATURATION	0x0D	饱和度 (彩色相机支持)	R/W
TUIDP_CLRTEMPERATURE	0x0E	色温	R/W
TUIDP_CLRMATRIX	0x0F	色彩矩阵	R/W
TUIDP_DPCLEVEL	0x10	坏点校正	R/W
TUIDP_BLACKLEVELHG	0x11	黑电平高增益	R/W
TUIDP_BLACKLEVELLG	0x12	黑电平低增益	R/W
TUIDP_POWEEFREQUENCY	0x13	消频闪(50HZ 或 60HZ)	R/W

TUIDP_HUE	0x14	色调	R/W
TUIDP_LIGHT	0x15	亮度	R/W
TUIDP_ENHANCE_STRENGTH	0x16	增强（通透性）	R/W
TUIDP_NOISELEVEL_3D	0x17	3D 降噪	R/W
TUIDP_FOCUS_POSITION	0x18	对焦位置	R/W
TUIDP_FRAME_RATE	0x19	帧率	R/W
TUIDP_START_TIME	0x1A	GPS 开始时间	R/W
TUIDP_FRAME_NUMBER	0x1B	GPS 采集张数	R/W
TUIDP_INTERVAL_TIME	0x1C	GPS 间隔时间	R/W
TUIDP_GPS_APPLY	0x1D	GPS 使能开关	R/W
TUIDP_AMB_TEMPERATURE	0x1E	环境温度（外接设备传入）	R/W
TUIDP_AMB_HUMIDITY	0x1F	环境湿度（外接设备传入）	R/W
TUIDP_AUTO_CTRLTEMP	0x20	自动控制温度使能	R/W
TUIDP_AVERAGEGRAY	0x21	平均灰度值设置	R/W
TUIDP_AVERAGEGRAYTHD	0x22	平均灰度值阈值	R/W
TUIDP_ENHANCETHD	0x23	增强阈值设置	R/W
TUIDP_ENHANCEPARA	0x24	增强参数设置	R/W
TUIDP_EXPOSUREMAX	0x25	最大曝光时间设置	R/W
TUIDP_EXPOSUREMIN	0x26	最小曝光时间设置	R/W
TUIDP_GAINMAX	0x27	最大增益设置	R/W
TUIDP_GAINMIN	0x28	最小增益设置	R/W
TUIDP_THROUGHFOGPARA	0x29	透雾参数设置	R/W
TUIDP_ENDPROPERTY	0x2A	属性 ID 结束位	R/W

5.1.5 TUCAM_IDPPROP 图像拼接代码

名称	代码	描述
TUIDPP_EDF_QUALITY	0x00	景深融合质量（1：低质量 2：中质量 3：高质量）
TUIDPP_STITCH_SPEED	0x01	图像拼接速度（0：快速 1：正常）
TUIDPP_STITCH_BGC_RED	0x02	图像拼接背景红色通道值
TUIDPP_STITCH_BGC_GREEN	0x03	图像拼接背景绿色通道值
TUIDPP_STITCH_BGC_BLUE	0x04	图像拼接背景蓝色通道值
TUIDPP_STITCH_VALID	0x05	图像拼接结果无效（只读）
TUIDPP_STITCH_AREA_X	0x06	图像拼接结果当前 X 坐标值（只读）
TUIDPP_STITCH_AREA_Y	0x07	图像拼接结果当前 Y 坐标值（只读）
TUIDPP_STITCH_NEXT_X	0x08	图像拼接结果下一个 X 坐标值（只读）
TUIDPP_STITCH_NEXT_Y	0x09	图像拼接结果下一个 Y 坐标值（只读）
TUIDPP_ENDPPROPERTY	0x0A	图片拼接属性 ID 结束位

5.1.6 TUCAM_IDCROI 区域计算代码

名称	代码	描述
TUIDCR_WBALANCE	0x00	计算区域白平衡
TUIDCR_BBALANCE	0x01	计算区域黑平衡
TUIDCR_BLOFFSET	0x02	计算区域黑电平偏移
TUIDCR_FOCUS	0x03	计算区域对焦
TUIDCR_EXPOSURETM	0x04	计算区域曝光时间
TUIDCR_END	0x05	计算区域 ID 结束位

5.1.7 TUCAM_CAPTURE_MODES 捕获模式代码

名称	代码	描述
TUCCM_SEQUENCE	0x00	采用序列模式（流模式），捕获连续的图像数据
TUCCM_TRIGGER_STANDARD	0x01	采用标准触发模式
TUCCM_TRIGGER_SYNCHRONOUS	0x02	采用同步触发模式
TUCCM_TRIGGER_GLOBAL	0x03	采用全局触发模式
TUCCM_TRIGGER_SOFTWARE	0x04	采用软件触发模式
TUCCM_TRIGGER_GPS	0x05	采用 GPS 触发模式

5.1.8 TUIMG_FORMATS 图像格式代码

名称	代码	描述
TUFMT_RAW	0x01	保存图像为 RAW 格式。Tucsen 自定义的一种相机图像文件格式（包含头部 + 图像数据）
TUFMT_TIF	0x02	保存图像为 TIF 格式，也叫 TIF 格式。支持不同颜色模式、路径、透明度和通道，是打印文档最常用的格式。 优点：保存丰富的图像层次和细节，画面质量无损。 缺点：占用存储空间大。
TUFMT_PNG	0x04	保存图像为 PNG 格式。 优点：支持高级别无损压缩；支持透明背景。 缺点：对旧浏览器和软件兼容性较差。
TUFMT_JPG	0x08	保存图像为 JPG 格式。 优点：能将图片压缩至很小的存储空间，对色彩信息的保留较好。 缺点：有损压缩会降低图片数据质量。
TUFMT_BMP	0x10	保存图像为 BMP 格式。Windows 系统中标准图像文件格式，

		支持多种 Windows 应用程序使用。 优点：无损压缩；图像画质优秀。 缺点：占用存储空间大。
--	--	--

5.1.9 TUCAM_TRIGGER_EXP 触发曝光时间模式代码

名称	代码	描述
TUCTE_EXPTM	0x00	接收到触发信号后，由 TUIDP_EXPOSURETM 设置的曝光时间决定
TUCTE_WIDTH	0x01	接收到触发信号后，曝光时间由电平的宽度所决定

5.1.10 TUCAM_TRIGGER_EDGE 激发边沿类型代码

名称	代码	描述
TUCTD_RISING	0x00	接收到的触发电平处于上升沿时开始曝光
TUCTD_FALLING	0x01	接收到的触发电平处于下降沿时开始曝光

5.1.11 TUCAM_TRIGGER_READOUTDIRRESET 触发读取输出复位代码

名称	代码	描述
TUCTD_YES	0x00	重置
TUCTD_NO	0x01	不重置

5.1.12 TUCAM_TRIGGER_READOUTDIR 触发输出方向代码

名称	代码	描述
TUCTD_DOWN	0x00	向下
TUCTD_UP	0x01	向上
TUCTD_DOWNUPCYC	0x02	向下循环

5.1.13 TUFPM_FORMATS 帧格式代码

名称	代码	描述
TUFPM_FMT_RAW	0x10	RAW 格式数据
TUFPM_FMT_USUAL	0x11	通常格式数据（8bit/16bit、黑白/彩色）

TUFRM_FMT_RGB888	0x12	RGB888 格式数据（可用于显示）
------------------	------	--------------------

5.1.14 TUCAM_OUTPUTTRG_PORT 触发输出端口代码

名称	代码	描述
TUPORT_ONE	0x00	使用端口 1
TUPORT_TWO	0x01	使用端口 2
TUPORT_THREE	0x02	使用端口 3

5.1.15 TUCAM_OUTPUTTRG_KIND 触发输出种类代码

名称	代码	描述
TUOPT_GND	0x00	低电平
TUOPT_VCC	0x01	高电平
TUOPT_IN	0x02	触发输入
TUOPT_EXPSTART	0x03	曝光开始参考点
TUOPT_EXPGLOBAL	0x04	全局曝光
TUOPT_READEND	0x05	读出结束

5.1.16 TUCAM_OUTPUTTRG_EDGE 触发输出边沿代码

名称	代码	描述
TUOPT_RISING	0x00	触发上升边沿
TUOPT_FAILING	0x01	触发下降边沿

5.2 结构体

一些函数需要一个指向结构体的指针作为参数，对于结构体参数，结构体中有[in]标识变量，意味着应用程序必须在调用之前设置值，有[out]标识变量，意味着函数返回时填充一个值。

5.2.1 初始化

```
1. class TUCAM_INIT(Structure):  
2.     _fields_ = [
```

```
3.         ("uiCamCount",      c_uint32),
4.         ("pstrConfigPath", c_char_p)
5.     ]
```

接口 TUCAM_API_Init 输入参数：

名称	描述
uiCamCount	[out] 返回当前连接的相机个数
pstrConfigPath	[in] 输入保存相机参数的路径，当执行 TUCAM_File_SaveProfiles 函数时保存该路径下

5.2.2 打开相机

```
1. class TUCAM_OPEN(Structure):
2.     _fields_ = [
3.         ("uidxOpen",      c_uint32),
4.         ("hldxTUCam",     c_void_p)
5.     ]
```

接口 TUCAM_Dev_Open 输入参数：

名称	描述
uidxOpen	[in] 打开指定索引相机
hldxTUCam	[out] 已打开相机的句柄

5.2.3 打开图像

```
1. class TUIMG_OPEN(Structure):
2.     _fields_ = [
3.         ("pszfileName",   c_void_p),
4.         ("hldxTUImg",     c_void_p)
5.     ]
```

接口 TUIMG_File_Open 输入参数：

名称	描述
pszfileName	[in] 打开指定索引相机
hIdxTUImg	[out] 已打开图像的句柄

5.2.4 相机信息

```

1. class TUCAM_VALUE_INFO(Structure):
2.     _fields_ = [
3.         ("nID", c_int32),
4.         ("nValue", c_int32),
5.         ("pText", c_char_p),
6.         ("nTextSize", c_int32)
7.     ]

```

接口 TUCAM_Get_Info \ TUCAM_Get_InfoEx 输入参数：

名称	描述
nID	[in] 设备信息 ID,参考 TUCAM_IDINFO
nValue	[in] 设备信息的值
pText	[in / out] 指向设备信息文本的指针
nTextSize	[in] 文本缓存大小

5.2.5 性能的属性

```

1. class TUCAM_CAPA_ATTR(Structure):
2.     _fields_ = [
3.         ("idCapa", c_int32),
4.         ("nValMin", c_int32),
5.         ("nValMax", c_int32),
6.         ("nValDft", c_int32),
7.         ("nValStep", c_int32)
8.     ]

```

接口 TUCAM_Capa_GetAttr 输入参数：

名称	描述
idCapa	[in] 相机性能 ID,参考 TUCAM_IDCAPA
nValMin	[out] 可取最小值

nValMax	[out] 可取最大值
nValDft	[out] 设备默认值
nValStep	[out] 有效值范围内，最小步进

5.2.6 属性的属性

```

1. class TUCAM_PROP_ATTR(Structure):
2.     _fields_ = [
3.         ("idProp",    c_int32),
4.         ("nIdxChn",   c_int32),
5.         ("dbValMin",  c_double),
6.         ("dbValMax",  c_double),
7.         ("dbValDft",  c_double),
8.         ("dbValStep", c_double)
9.     ]

```

接口 TUCAM_Prop_GetAttr 输入参数：

名称	描述
idProp	[in] 属性 ID，参考 TUCAM_IDPROP
nIdxChn	[in/out] 当前通道索引，如果使用默认通道手动设置 0。
dbValMin	[out] 可取最小值
dbValMax	[out] 可取最大值
dbValDft	[out] 设备默认值
dbValStep	[out] 可取范围内最小步进

5.2.7 性能 / 属性值文本

```

1. class TUCAM_VALUE_TEXT(Structure):
2.     _fields_ = [
3.         ("nID",      c_int32),
4.         ("dbValue",  c_double),
5.         ("pText",    c_char_p),
6.         ("nTextSize", c_int32)
7.     ]

```

接口 TUCAM_Capa_GetValueText 和 TUCAM_Prop_GetValueText 输入参数：

名称	描述
nID	[in] 属性或性能 ID，参考 TUCAM_IDPROP / TUCAM_IDCAPA 值。
dbValue	[in] 性能或属性有效值
pText	[in/out] 指向值文本指针
nTextSize	[out] 值文本缓存大小

5.2.8 ROI 的属性

```

1. class TUCAM_ROI_ATTR(Structure):
2.     _fields_ = [
3.         ("bEnable",    c_int32),
4.         ("nHOffset",   c_int32),
5.         ("nVOffset",   c_int32),
6.         ("nWidth",     c_int32),
7.         ("nHeight",    c_int32)
8.     ]

```

接口 TUCAM_Cap_SetROI 和 TUCAM_Cap_GetROI 输入参数：

名称	描述
bEnable	[in/out] ROI 使能
nHOffset	[in/out] 水平方向偏移量，以像素为单位
nVOffset	[in/out] 垂直方向偏移量，以像素为单位
nWidth	[in/out] ROI 宽度
nHeight	[in/out] ROI 高度

5.2.9 区域计算属性

```

9. class TUCAM_CALC_ROI_ATTR(Structure):
10.     _fields_ = [
11.         ("bEnable",    c_int32),
12.         ("idCalc",     c_int32),
13.         ("nHOffset",   c_int32),
14.         ("nVOffset",   c_int32),
15.         ("nWidth",     c_int32),
16.         ("nHeight",    c_int32)
17.     ]

```

接口 TUCAM_Calc_SetROI 和 TUCAM_Calc_GetROI 输入参数：

名称	描述
bEnable	[in/out] ROI 使能
idCalc	[in] 区域计算类型，参考 TUCAM_IDCROI
nHOffset	[in/out] 水平方向偏移量，以像素为单位
nVOffset	[in/out] 垂直方向偏移量，以像素为单位
nWidth	[in/out] ROI 宽度
nHeight	[in/out] ROI 高度

5.2.10 触发的属性

```

1. class TUCAM_TRIGGER_ATTR(Structure):
2.     _fields_ = [
3.         ("nTgrMode",      c_int32),
4.         ("nExpMode",      c_int32),
5.         ("nEdgeMode",     c_int32),
6.         ("nDelayTm",      c_int32),
7.         ("nFrames",       c_int32),
8.         ("nBufFrames",    c_int32)
9.     ]

```

接口 TUCAM_Cap_SetTrigger 和 TUCAM_Cap_GetTrigger 输入参数：

名称	描述
nTgrMode	[in/out] 触发模式参考值 TUCAM_CAPTURE_MODES
nExpMode	[in/out] 曝光模式[0, 1], 0: 曝光时间, 1: 电平宽度, 参考 TUCAM_TRIGGER_EXP
nEdgeMode	[in/out] 边沿激发模式[0, 1], 0: 上升沿, 1: 下降沿, 参考 TUCAM_TRIGGER_EDGE
nDelayTm	[in/out] 触发延迟时间，单位毫秒
nFrames	[in/out] 一次触发输出帧数
nBufFrames	[in/out] 缓冲区中有多少帧

5.2.11 触发输出的属性

```

1. class TUCAM_TRGOUT_ATTR(Structure):
2.     _fields_ = [
3.         ("nTgrOutPort",      c_int32),
4.         ("nTgrOutMode",      c_int32),
5.         ("nEdgeMode",        c_int32),
6.         ("nDelayTm",         c_int32),
7.         ("nWidth",           c_int32)
8.     ]

```

接口 TUCAM_Cap_SetTriggerOut 和 TUCAM_Cap_GetTriggerOut 输入参数：

名称	描述
nTgrOutPort	[in/out] 触发输出口，参考 TUCAM_OUTPUTTRG_PORT
nTgrOutMode	[in/out] 触发输出模式，参考 TUCAM_OUTPUTTRG_KIND
nEdgeMode	[in/out] 边沿激发模式
nDelayTm	[in/out] 触发输出延迟时间
nWidth	[in/out] 触发输出脉宽

5.2.12 任意 Bin 属性

```

1. class TUCAM_BIN_ATTR(Structure):
2.     _fields_ = [
3.         ("bEnable",      c_int32),
4.         ("nMode",        c_int32),
5.         ("nWidth",       c_int32),
6.         ("nHeight",      c_int32)
7.     ]

```

接口 TUCAM_Cap_SetBIN 和 TUCAM_Cap_GetBIN 输入参数：

名称	描述
bEnable	[in/out] 任意 bin 使能
nWidth	[in/out] 任意 bin 宽度
nHeight	[in/out] 任意 bin 高度

5.2.13 帧结构

```

1. class TUCAM_FRAME(Structure):
2.     _fields_ = [
3.         ("szSignature", c_char * 8),
4.         ("usHeader", c_ushort),
5.         ("usOffset", c_ushort),
6.         ("usWidth", c_ushort),
7.         ("usHeight", c_ushort),
8.         ("uiWidthStep", c_uint),
9.         ("ucDepth", c_ubyte),
10.        ("ucFormat", c_ubyte),
11.        ("ucChannels", c_ubyte),
12.        ("ucElemBytes", c_ubyte),
13.        ("ucFormatGet", c_ubyte),
14.        ("uiIndex", c_uint),
15.        ("uiImgSize", c_uint),
16.        ("uiRsdSize", c_uint),
17.        ("uiHstSize", c_uint),
18.        ("pBuffer", c_void_p)
19.    ]
20.

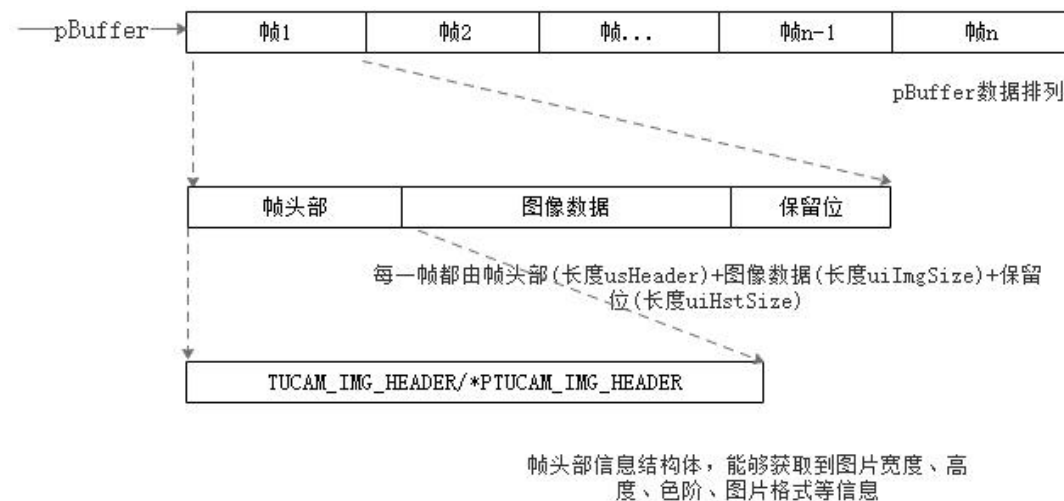
```

TUCAM_Buf_WaitForFrame 输入参数:

名称	描述
szSignature[8]	[out] 版权信息
usHeader	[out] 帧头部大小
usOffset	[out] 帧数据偏移大小
usWidth	[out] 水平像素数
usHeight	[out] 垂直像素数
uiWidthStep	[out] 帧图像的宽度步长
ucDepth	[out] 帧图像数据位深
ucFormat	[out] 帧图像数据格式
ucChannels	[out] 帧图像的通道数
ucElemBytes	[out] 帧图像的数据字节数
ucFormatGet	[in/out] 需要获取的图像格式。参考 TUFrm_FORMATS
uiIndex	[out] 帧图像序列号(保留)
uiImgSize	[out] 帧图像数据的大小
uiRsdSize	[in] 需要获取的帧数

uiHstSize	[out] 帧图像保留字段
pBuffer	[out] 指向帧数据缓存的指针

pBuffer 数据排列如下，可以根据帧头部大小获取到每一帧的头部信息，也可以根据帧数据偏移获取到图像数据，进行绘制展示：



5.2.14 文件保存

```

1. class TUCAM_FILE_SAVE(Structure):
2.     _fields_ = [
3.         ("nSaveFmt", c_int32),
4.         ("pstrSavePath", c_char_p),
5.         ("pFrame", POINTER(TUCAM_FRAME))
6.     ]

```

TUCAM_File_SaveImage 图像保存用到这个结构体：

名称	描述
nSaveFmt	[in] 保存图像的格式 TUIMG_FORMATS
pstrSavePath	[in] 保存图像路径(不包括扩展名)
pFrame	[in] 指向帧数据指针

5.2.15 录像保存

```

1. class TUCAM_REC_SAVE(Structure):
2.     _fields_ = [
3.         ("nCodec", c_int32),

```

```
4.         ("pstrSavePath", c_char_p),
5.         ("fFps",          c_float)
6.     ]
```

接口 TUCAM_Rec_Start 输入参数:

名称	描述
nCodec	[in] 编码解码类型, 默认为 0
pstrSavePath	[in] 保存路径(包含文件名)
fFps	[in] 需要保存的帧率

5.2.16 寄存器读写

```
1. class TUCAM_REG_RW(Structure):
2.     _fields_ = [
3.         ("nRegType",    c_int32),
4.         ("pBuf",        c_char_p),
5.         ("nBufSize",    c_int32)
6.     ]
7.
```

接口 TUCAM_Reg_Read 和 TUCAM_Reg_Write 输入参数:

名称	描述
nRegType	[in] 读写寄存器类型, 参考 TUREG_TYPE
pBuf	[in/out] 指向读写内容缓冲区指针
nBufSize	[in] 缓冲区大小

5.3 函数

5.3.1 API 初始化 / 反初始化

5.3.1.1 TUCAM_Api_Init 和 TUCAM_Api_Uninit

描述

卸载 TUCAM-API 库, 包括释放驱动绑定和一些内部资源。整个程序结束时调用一次。

声明

TUCAMRET TUCAM_Api_Uninit ()

参数

无输入参数

错误代码

TUCAMRET_NOT_INIT

TUCAM-API 未初始化

相关接口

TUCAM_Api_Uninit

5.3.2 打开、关闭相机

5.3.2.1 TUCAM_Dev_Open

描述

打开相机，调用后相机处于工作模式，可以响应其他接口的调用。在此之前要保证有相机存在，即要在调用 TUCAM_Api_Init 初始化之后。

声明

TUCAMRET TUCAM_Dev_Open(pointer(TUCAMOPEN))

参数

TUCAMOPEN =

打开相机类，参考结构体 TUCAM_OPEN

TUCAM_OPEN(0, 0)

错误代码

TUCAMRET_NOT_INIT

TUCAM-API 未初始化

TUCAMRET_INVALID_PARAM

无效参数，当 pOpenParam 指针为空时

TUCAMRET_OUT_OF_RANGE

超出范围，当需要打开的相机索引号超出连接相机的范围时

TUCAMRET_FAILOPEN_CAMER

打开相机失败

A

TUCAMRET_INVALID_CAMERA

无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Close

5.3.2.2 TUCAM_Dev_Close

描述

关闭相机，调用后相机处于待机状态，不响应其他接口的调用。

声明

TUCAMRET TUCAM_Dev_Close(TUCAMOPEN.hIdxTUCam)

参数

无输入参数

错误代码

TUCAMRET	TUCAM-API 未初始化
----------	----------------

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open

5.3.2.3 TUCAM_Dev_GetInfo

描述

获取相机的相关信息，如 USB 口类型，相机产品号，API 版本号、固件版本号、相机类型等。在此之前要保证有相机存在，并且保证相机打开，即要在调用 TUCAM_Api_Init 初始化和调用 TUCAM_Api_Open 之后。

声明

TUCAMRET TUCAM_Dev_GetInfo(hTUCam, pointer(info))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_VALUE_INFO info	相机信息值类，参考 TUCAM_VALUE_INFO

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_PARAM	无效参数，当产品信息代码不存在时，参考 TUCAM_IDINFO
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close

5.3.2.4 TUCAM_Dev_GetInfoEx**描述**

获取相机的相关信息，如 USB 口类型，相机产品号，API 版本号、相机类型等。在此之前要保证有相机存在，并且保证接口初始化，即只需要在调用 TUCAM_Api_Init 之后，无需调用 TUCAM_Api_Open。仅支持 TUIDI_CAMERA_MODEL、TUIDI_BUS、TUIDI_VENDOR、TUIDI_PRODUCT、TUIDI_VERSION_API、TUIDI_BCDDEVICE。

声明

TUCAMRET TUCAM_Dev_GetInfoEx (UINT32 uilCam, pointer(info))

参数

UINT32 uilCam	相机的索引号
TUCAM_VALUE_INFO info	相机信息值类，参考 TUCAM_VALUE_INFO

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_PARAM	无效参数,当产品信息代码不存在时,参考 TUCAM_IDINFO
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit

5.3.3 性能获取和设置**5.3.3.1 TUCAM_Capa_GetAttr****描述**

获取性能参数的属性值。获取的属性包含该参数的最小值、最大值、默认值和步长。具体支持的性能类型参考 TUCAM_IDCAPA。

声明

TUCAMRET TUCAM_Capa_GetAttr (hTUCam, pointer(capaAttr))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_CAPA_ATTR capaAttr	相机性能属性类，参考 TUCAM_CAPA_ATTR

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDCAPA	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDCAPA
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时
TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Capa_GetValue、TUCAM_Capa_SetValue、TUCAM_Capa_GetValueText

5.3.3.2 TUCAM_Capa_GetValue

描述

获取性能参数的当前属性值。具体支持的性能类型参考 TUCAM_IDCAPA。

声明

TUCAMRET TUCAM_Capa_GetValue (hTUCam, capa, pointer(value))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_IDCAPA capa	相机性能属性 ID，参考 TUCAM_IDCAPA
value = c_int(0)	返回当前值

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDCAPA	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDCAPA
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时
TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Capa_GetAttr、TUCAM_Capa_SetValue、TUCAM_Capa_GetValueText

5.3.3.3 TUCAM_Capa_SetValue

描述

设置性能参数的当前属性值。具体支持的性能类型参考 TUCAM_IDCAPA。

声明

TUCAMRET TUCAM_Capa_SetValue (hTUCam, capa, value)

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_IDCAPA capa	相机性能属性 ID，参考 TUCAM_IDCAPA
value=c_int(0)	需要设置的当前值

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDCAPA	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDCAPA
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时
TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Capa_GetAttr、TUCAM_Capa_GetValue、TUCAM_Capa_GetValueText

5.3.3.4 TUCAM_Capa_GetValueText

描述

获取性能参数的当前属性值的文本信息。具体支持的性能类型参考 TUCAM_IDCAPA。

声明

TUCAMRET TUCAM_Capa_GetValueText (hTUCam, pointer(text))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_VALUE_TEXT text	获取性能参数的文本信息类，TUCAM_VALUE_TEXT

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_FAILURE	文本缓冲区大小为 0 或 pText 指针为空时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit

TUCAM_Dev_Open、TUCAM_Dev_Close

TUCAM_Capa_GetAttr、TUCAM_Capa_SetValue、TUCAM_Capa_GetValue

5.3.4 属性获取和设置

5.3.4.1 TUCAM_Prop_GetAttr

描述

获取属性参数的属性值。获取的属性包含该参数的最小值、最大值、默认值和步长。具体支持的性能类型参考 TUCAM_IDPROP。

声明

TUCAMRET TUCAM_Prop_GetAttr (hTUCam, propAttr)

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_PROP_ATTR propAttr	相机性能属性类，参考 TUCAM_PROP_ATTR

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDPROP	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDPROP
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时
TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_OUT_OF_RANGE	需要获取的通道超出范围时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit

TUCAM_Dev_Open、TUCAM_Dev_Close

TUCAM_Prop_GetValue、TUCAM_Prop_SetValue、TUCAM_Prop_GetValueText

5.3.4.2 TUCAM_Prop_GetValue

描述

获取属性参数的当前属性值。具体支持的性能类型参考 TUCAM_IDPROP。

声明

TUCAMRET TUCAM_Prop_GetValue (HDTUCAM hTUCam, prop, pointer(value), nChn);

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_IDPROP prop	相机属性的 ID，参考 TUCAM_IDPROP
value = c_double(0)	返回当前值
nChn = c_int(0)	需要获取的当前通道（默认为 0，黑白相机为 0）

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDPROP	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDPROP
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时
TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_OUT_OF_RANGE	需要获取的通道超出范围时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Prop_GetAttr、TUCAM_Prop_SetValue、TUCAM_Prop_GetValueText

5.3.4.3 TUCAM_Prop_SetValue

描述

设置属性参数的当前属性值。具体支持的性能类型参考 TUCAM_IDPROP。

声明

TUCAMRET TUCAM_Prop_SetValue (hTUCam, prop, value, nChn)

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_IDPROP prop	相机属性的 ID，参考 TUCAM_IDPROP
value = c_double(0)	需要设置的当前值
nChn = c_int(0)	需要获取的当前通道（默认为 0，黑白相机为 0）

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_IDPROP	无效的性能代码，当性能代码不存在时，参考 TUCAM_IDPROP
TUCAMRET_INVALID_VALUE	无效的值，当 pAttr 指针为空时

TUCAMRET_NOT_SUPPORT	当底层请求不支持时
TUCAMRET_OUT_OF_RANGE	需要获取的通道超出范围时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Prop_GetAttr、TUCAM_Prop_GetValue、TUCAM_Prop_GetValueText

5.3.4.4 TUCAM_Prop_GetValueText

描述

获取属性参数的当前属性值的文本信息。具体支持的性能类型参考 TUCAM_IDPROP。

声明

TUCAMRET TUCAM_Prop_GetValueText (hTUCam, pointer(text))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_VALUE_TEXT text	获取性能参数的文本信息类，TUCAM_VALUE_TEXT

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_FAILURE	文本缓冲区大小为 0 或 pText 指针为空时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Prop_GetAttr、TUCAM_Prop_SetValue、TUCAM_Prop_GetValue

5.3.5 内存管理

5.3.5.1 TUCAM_Buf_Alloc

描述

分配内存空间用于数据捕获。当应用程序调用这个接口时，SDK 分配必要的内部缓冲区来缓冲图像采集。捕获不从这一时刻开始。开始采集，应用程序必须调用 TUCAM_Cap_Start 接口。如果缓冲区不再是必要的，应用程序应该调用 TUCAM_Buf_Release 接口来释放内部缓冲区。

声明

TUCAMRET TUCAM_Buf_Alloc (hTUCam, pointer(frame))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_FRAME frame	图像帧结构类，参考 TUCAM_FRAME

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_PARAM	当 frame 指针为空时
TUCAMRET_EXCLUDED	当 TUCAM_Buf_Alloc 被调用，且未释放时
TUCAMRET_OUT_OF_RANGE	当需要获取的帧数超出范围时
TUCAMRET_NO_MEMORY	当内存不足时
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame、TUCAM_Buf_CopyFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.2 TUCAM_Buf_Release

描述

释放用于数据捕获的内存空间。如果在捕获过程中调用该接口，这个接口将返回相机处于繁忙的状态。

声明

TUCAMRET TUCAM_Buf_Release (hTUCam)

参数

HDTUCAM hTUCam	相机的句柄
----------------	-------

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_BUSY	相机处于繁忙状态
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close

TUCAM_Buf_Alloc
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame、TUCAM_Buf_CopyFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.3 TUCAM_Buf_AbortWait

描述

用于停止数据捕获时的等待。调用 TUCAM_Buf_WaitForFrame 进行数据捕获等待之后，使用该接口终止等待。

声明

TUCAMRET TUCAM_Buf_AbortWait (hTUCam);

参数

HDTUCAM hTUCam 相机的句柄

错误代码

TUCAMRET_NOT_INIT TUCAM-API 未初始化
TUCAMRET_INVALID_CAMERA 无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_WaitForFrame、TUCAM_Buf_CopyFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.4 TUCAM_Buf_WaitForFrame

描述

用于等待数据捕获完成。通过调用 TUCAM_Buf_Alloc 分配来的空间，来获取捕获到的帧数据。必须在调用 TUCAM_Cap_Start 开始捕获之后使用，否则会返回未准备的状态。

该函数属于阻塞函数，直至数据捕获完成或调用 TUCAM_Buf_AbortWait 终止。

帧结构体中 uiRsdSize 设置需要捕获的帧数，针对触发模式有效。例如：当触发一次需要返回 5 帧时，该函数等待 5 帧数据都捕获结束之后才返回。

注意：返回的帧结构 pBuffer 的数据排列，是帧头部(usHeader)+图像数据(uiImgSize) + 保留位(uiHstSize)。如果是多帧返回则按此顺序往后排列。

声明

TUCAMRET TUCAM_Buf_WaitForFrame(hTUCam, pointer(frame), timeOut)

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_FRAME frame	帧结构体类对象
timeOut = c_in(1000)	超时退出的时间，默认 1S

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_READY	当未调用 TUCAM_Cap_Start 开始捕获时
TUCAMRET_NO_MEMORY	当未调用 TUCAM_Buf_Alloc 创建内存空间时
TUCAMRET_NO_RESOURCE	当 frame 指针为空时
TUCAMRET_OUT_OF_RANGE	当获取的帧数大于 1 时且获取的格式和 TUCAM_Buf_Alloc 不同
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_CopyFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.5 TUCAM_Buf_CopyFrame

描述

用于等待数据捕获完成之后拷贝不同于 TUCAM_Buf_Alloc 分配的图像格式数据。必须在 TUCAM_Buf_WaitForFrame 返回之后调用，否则无法获取正确的图像数据。

例如：分配的图像格式为 TUFRM_FMT_RGB888，通过该函数可以拷贝其他格式的数据（如 TUFRM_FMT_RAW），此接口无法拷贝大于 1 帧的数据即帧结构中的 uiRsdSize 不能大于 1。

注意：返回的帧结构 pBuffer 的数据排列，是帧头部(usHeader)+图像数据(uiImgSize) + 保留位(uiHstSize)。不支持多帧数据返回。

声明

TUCAMRET TUCAM_Buf_CopyFrame (hTUCam, pointer(frame))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_FRAME frame	帧结构体的类对象

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_READY	当未调用 TUCAM_Cap_Start 开始捕获时

TUCAMRET_NO_MEMORY	当未调用 TUCAM_Buf_Alloc 创建内存空间时
TUCAMRET_NO_RESOURCE	当 frame 指针为空时
TUCAMRET_OUT_OF_RANGE	当获取的帧数大于 1 时且获取的格式和 TUCAM_Buf_Alloc 不同
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.6 TUCAM_Buf_DataCallBack

描述

用于获取原始数据流注册回调函数，注册回调之后通过调用 TUCAM_Buf_Alloc 分配来的空间，来获取捕获到的帧数据。必须在调用 TUCAM_Cap_Start 开始捕获之后使用 TUCAM_Buf_GetData 函数便可获取原始数据流。

声明

TUCAMRET TUCAM_Buf_DataCallBack(hTUCam, CALL_BACK_FUN, CALL_BACK_USER)

参数

HDTUCAM hTUCam	相机的句柄
CALL_BACK_FUN	获取原始数据流回调函数
CALL_BACK_USER	用户数据回调函数注册

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_GetData
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.5.7 TUCAM_Buf_GetData

描述

用于获取原始数据流函数，当使用 TUCAM_Buf_DataCallBack 函数。

注册回调之后通过调用 TUCAM_Buf_Alloc 分配来的空间，来获取捕获到的帧数据。必须在调用 TUCAM_Cap_Start 开始捕获之后使用。

声明

TUCAMRET TUCAM_Buf_GetData (hTUCam, pointer(frame))

参数

HDTUCAM hTUCam	相机的句柄
frame =	原始帧结构体类对象
TUCAM_RAWIMG_HEADER()	

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_FAILURE	获取帧失败
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_DataCallBack
TUCAM_Cap_Start、TUCAM_Cap_Stop

5.3.6 捕获控制

5.3.6.1 TUCAM_Cap_SetROI

描述

用于设定图像的感兴趣区域，以左上角为坐标原点。设置的水平偏移量、垂直偏移量、宽度和高度必须为 4 的倍数。（11BIT 模式下宽度大于 32，宽度乘高度必须为 32 的倍数）

声明

TUCAMRET TUCAM_Cap_SetROI(hTUCam, roiAttr)

参数

HDTUCAM hTUCam	相机的句柄
roiAttr= TUCAM_ROI_ATTR()	roiAttr 属性类对象

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_SUPPORT	不支持 ROI 设置

TUCAMRET_INVALID_CAMERA 无效的相机，相机句柄不存在时

相关接口

TUCAM_API_INIT、TUCAM_API_UNINIT
TUCAM_DEV_OPEN、TUCAM_DEV_CLOSE
TUCAM_BUF_ALLOC、TUCAM_BUF_RELEASE
TUCAM_BUF_ABORTWAIT、TUCAM_BUF_WAITFORFRAME
TUCAM_CAP_START、TUCAM_CAP_STOP
TUCAM_CAP_GETROI

5.3.6.2 TUCAM_Cap_GetROI

描述

用于设定图像的感兴趣区域，以左上角为坐标原点。设置的水平偏移量、垂直偏移量、宽度和高度必须为 4 的倍数。（11BIT 模式下宽度大于 32,宽度乘高度必须为 32 的倍数）

捕获不从这一时刻开始。开始采集调用 TUCAM_Cap_Start 接口之后。

声明

TUCAMRET TUCAM_Cap_GetROI (hTUCam, pointer(roiAttr))

参数

HDTUCAM hTUCam	相机的句柄
roiAttr = TUCAM_ROI_ATTR()	roiAttr 属性类对象

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_SUPPORT	不支持 ROI 设置
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Cap_SetROI

5.3.6.3 TUCAM_Cap_SetTrigger

描述

用于设定触发的属性。捕获不从这一时刻开始，开始采集调用 TUCAM_Cap_Start 接口之后。

曝光模式：

TUCTE_EXPTM

表示曝光时间由软件设定

TUCTE_WIDTH

表示曝光时间由输入电平宽度设定

激发边沿模式：

TUCTD_RISING

表示触发信号为上升沿有效

TUCTD_FALLING

表示触发信号为下降沿有效

帧数：表示接收一个触发信号后，拍摄多少张图像，每张图像的曝光时间是相同的，取决于软件设定。（选择电平宽度时，该功能无效。）

延迟：表示接收到一个触发信号后，可以设置多长的延迟时间才使相机进行触发曝光。

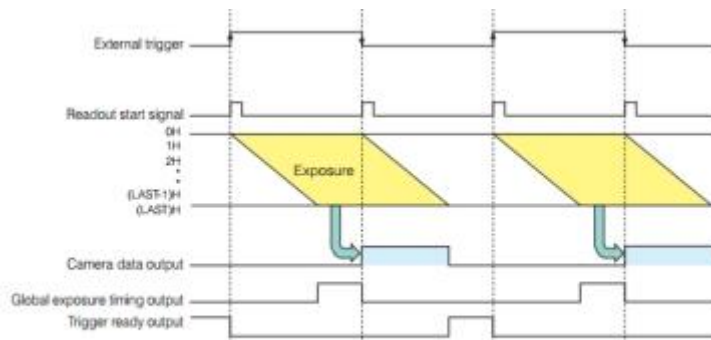


Fig. 19: Normal level trigger mode (rising edge)

触发模式支持的参数：

模式	TUCAM_TRIGGER_EXP	TUCAM_TRIGGER_EDGE	延时	帧数
标准触发	支持	支持	支持	支持
同步触发	支持	支持	不支持	不支持
全局触发	不支持	支持	不支持	不支持
软件触发	不支持	不支持	不支持	不支持

同步触发：即同步取图，第一次触发启动，第二次触发输出同步图像。

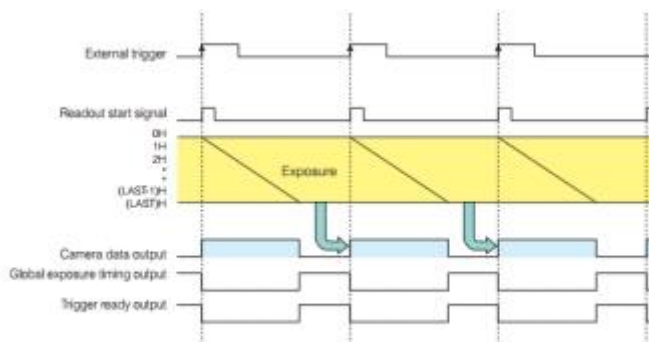
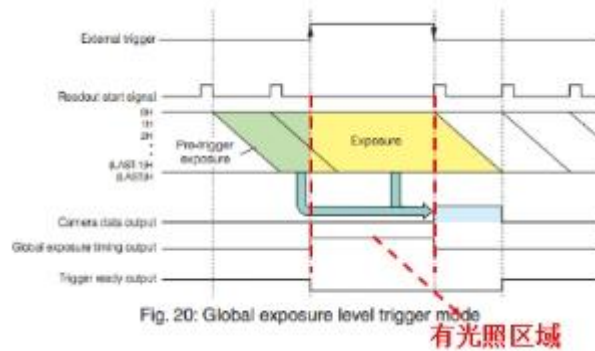


Fig. 21: Normal synchronous readout trigger mode (rising edge)

全局触发：一般用于光源可控的场景。



软件触发：通过软件下发命令模拟触发信号。

声明

TUCAMRET TUCAM_Cap_SetTrigger(hTUCam, tgrAttr)

参数

HDTUCAM hTUCam 相机的句柄
tgrAttr=TUCAM_TRIGGER_ATTR() 触发属性类对象

错误代码

TUCAMRET_NOT_INIT TUCAM-API 未初始化
TUCAMRET_INVALID_CAMERA 无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Cap_GetTrigger、TUCAM_Cap_DoSoftwareTrigger

5.3.6.4 TUCAM_Cap_GetTrigger

描述

用于获取触发的属性。

声明

TUCAMRET TUCAM_Cap_GetTrigger(hTUCam, pointer(tgrAttr))

参数

HDTUCAM hTUCam 相机的句柄
tgrAttr=TUCAM_TRIGGER_ATT() 触发属性类对象

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Cap_SetTrigger

5.3.6.5 TUCAM_Cap_DoSoftwareTrigger

描述

执行软件触发命令。

声明

TUCAMRET TUCAM_Cap_DoSoftwareTrigger(hTUCam)

参数

HDTUCAM hTUCam	相机的句柄
----------------	-------

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_FAILURE	执行触发命令失败
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Cap_SetTrigger

5.3.6.6 TUCAM_Cap_SetTriggerOut

描述

用于设定触发输出的属性。

声明

TUCAMRET TUCAM_Cap_SetTriggerOut(hTUCam, tgroutAttr)

参数

HDTUCAM hTUCam	相机的句柄
tgroutAttr=TUCAM_TRGOUT_AT	触发输出属性类的对象
TR()	

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_SUPPORT	不支持设置
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release

5.3.6.7 TUCAM_Cap_GetTriggerOut

描述

用于获取触发输出的属性。

声明

TUCAMRET TUCAM_Cap_GetTriggerOut(hTUCam, pointer(tgroutAttr))

参数

HDTUCAM hTUCam	相机的句柄
tgroutAttr=TUCAM_TRGOUT_AT	触发输出属性类对象
TR()	

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_SUPPORT	不支持设置
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release

5.3.6.8 TUCAM_Cap_Start

描述

开始进行数据捕获。在开始捕获之前，应使用 TUCAM_Buf_alloc 准备捕获缓冲区，如果使用 TUCAM_SEQUENCE 模式将持续捕获，直到调用 TUCAM_Cap_Stop 被调用。适用配置好感兴趣区域和触发模式。

声明

TUCAMRET TUCAM_Cap_Start(hTUCam, uiMode)

参数

HDTUCAM hTUCam	相机的句柄
uiMode = c_int(0)	相机捕获的模式，参考 TUCAM_CAPTURE_MODES

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_FAILOPEN_BULKIN	打开相机捕获失败
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start
TUCAM_Cap_SetTrigger、TUCAM_SetROI

5.3.6.9 TUCAM_Cap_Stop

描述

停止进行数据捕获。

声明

TUCAMRET TUCAM_Cap_Stop (hTUCam)

参数

HDTUCAM hTUCam	相机的句柄
----------------	-------

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
-------------------	----------------

TUCAMRET_FAILOPEN_BULKIN 打开相机捕获失败
TUCAMRET_INVALID_CAMERA 无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_AbortWait、TUCAM_Buf_WaitForFrame
TUCAM_Cap_Stop

5.3.7 文件管理

5.3.7.1 TUCAM_File_SaveImage

描述

对帧数据进行保存。

声明

TUCAMRET TUCAM_File_SaveImage (hTUCam, fileSave)

参数

HDTUCAM hTUCam 相机的句柄
fileSave=TUCAM_FILE_SAVE() 文件保存结构体

错误代码

TUCAMRET_NOT_INIT TUCAM-API 未初始化
TUCAMRET_INVALID_PARAM 输入的参数无效
TUCAMRET_INVALID_PATH 输入的路径不存在
TUCAMRET_FAILURE 文件保存失败
TUCAMRET_INVALID_CAMERA 无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_WaitForFrame、TUCAM_Buf_CopyFrame

5.3.7.2 TUCAM_Rec_Start

描述

打开录像文件，对帧数据进行录像保存，此时并未写入数据。设置的帧率需要大于 1fps，不足

1fps 的将按 1fps 来创建录像文件。

声明

TUCAMRET TUCAM_Rec_Start(hTUCam, recSave)

参数

HDTUCAM hTUCam	相机的句柄
recSave=TUCAM_REC_SAVE()	录像文件保存结构体

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_PARAM	输入的参数无效
TUCAMRET_INVALID_PATH	输入的路径不存在
TUCAMRET_FAILOPEN_FILE	文件打开失败
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Rec_Stop、TUCAM_Rec_AppendFrame

5.3.7.3 TUCAM_Rec_AppendFrame

描述

将图像数据写入文件，在 TUCAM_Buf_WaitForFrame 调用该接口。

声明

TUCAMRET TUCAM_Rec_AppendFrame(hTUCam, pointer(frame))

参数

HDTUCAM hTUCam	相机的句柄
TUCAM_FRAME frame	帧结构体对象

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NOT_READY	未调用 TUCAM_Rec_Start 接口
TUCAMRET_OUT_OF_RANGE	写入的图像宽度和高度与创建时不一致
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Rec_Start、TUCAM_Rec_Stop

5.3.7.4 TUCAM_Rec_Stop

描述

关闭录像文件，此时调用 TUCAM_Rec_AppendFrame 将无法写入数据。

声明

TUCAMRET TUCAM_Rec_Stop (hTUCam)

参数

HDTUCAM hTUCam	相机的句柄
----------------	-------

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Buf_Alloc、TUCAM_Buf_Release
TUCAM_Buf_WaitForFrame
TUCAM_Cap_Start、TUCAM_Cap_Stop
TUCAM_Rec_Start、TUCAM_Rec_AppendFrame

5.3.8 扩展控制

5.3.8.1 TUCAM_Reg_Read

描述

读取寄存器的内容。读取的类型参考 TUREG_TYPE。

声明

TUCAMRET TUCAM_Reg_Read (hTUCam, regRW)

参数

HDTUCAM hTUCam	相机的句柄
regRW=TUCAM_REG_RW()	寄存器读写结构类

错误代码

TUCAMRET_NOT_INIT	TUCAM-API 未初始化
TUCAMRET_NO_MEMORY	传入的缓冲区未分配内存空间
TUCAMRET_NOT_SUPPORT	不支持该类型读取
TUCAMRET_INVALID_IDPARAM	无效的类型，参考 TUREG_TYPE
TUCAMRET_INVALID_CAMERA	无效的相机，相机句柄不存在时

相关接口

TUCAM_Api_Init、TUCAM_Api_Uninit
TUCAM_Dev_Open、TUCAM_Dev_Close
TUCAM_Reg_Write

6. 常见问题解答（FAQ）

6.1 问题排查思路

1. 基于 SDK 开发的程序有异常，建议先运行 SamplePro 或是 Mosaic 软件，查看相应功能是否能够正常运行。
2. 如果 SamplePro 或 Mosaic 软件正常，而开发程序异常，建议重点排查二次开发的程序代码。
3. 如果 SamplePro 或 Mosaic 也异常，建议查看如下常见问题解决方法，看是否有帮助。
4. 如果以上排查都不能解决，建议请记录相关问题现象和保存图片数据、以及我们 SamplePro 或 Mosaic 与 SDK 的文件版本和产品版本号，联系本公司的技术支持同事获取帮助支持。

6.2 常见问题解决方法

问题 1：用 SDK 开发存图异常。

问题原因：TUCAM_FRAME 结构体中的 pBuffer 需要偏置到 usHeader 头部长度后才是真正的帧数据指针。

解决方法：pBuf = create_string_buffer(m_frame.uilmgSize)
pointerData = c_void_p(m_frame.pBuffer + m_frame.usHeader)
memmove(pBuf, pointerData, m_frame.uilmgSize)，即 pBuf 指针才是帧数据指针。

问题 2：用 TUCAM_Buf_GetData 获取到的 TUCAM_RAWIMG_HEADER 结构体帧数据偏移头部图像异常。

问题原因：TUCAM_RAWIMG_HEADER 中的 plmgData 是没有头部的，直接就是图像数据了，所以不必要偏移头部。

解决方法：直接从 plmgData 指针拿到的数据就是帧数据。

问题 3：设置垂直镜像返回 TUCAMRET_NO_RESOURCE 问题。

问题原因：由于垂直镜像是软件层做的，即需要先 CaptureStart 后才能设置生效。

解决方法：先调用 TUCAM_Cap_Start 后，再做垂直镜像。

问题 4：在 CaptureStart 后设置垂直镜像仍是无效。

问题原因：由于垂直镜像是软件层做的，只是针对 TUCAM_Buf_WaitForFrame 获取数据才生效，对 TUCAM_Buf_GetData 接口获取是原始图是无效。

解决方法：可通过 TUCAM_Buf_WaitForFrame 获取数据做显示。

问题 5：调用 TUCAM_Buf_Alloc 后报错（0x80000204）。

问题原因：由于多次重复调用了此接口。

解决方法：此接口只调用一次即可，如果还要调需先调用 TUCAM_Buf_Release 后再调。

问题 6：设置自动色阶图像不生效。

问题原因：直方图计算没有开启，并且直方图开启需要在 CaptureStart 后才能设置生效。另外一方面注意设置参数接口是否用错了（TUCAM_Capa_SetValue、TUCAM_Prop_SetValue）。

解决方法：先开启直方图，然后设置自动色阶参数即可。

问题 7：16bit 相机为何拿到帧灰度值最大值是 255。

问题原因：我们帧数据都是按最小字节单位数据输出，16bit 帧需要两个字节拼接成一个像素，小端排列即低字节在前高字节在后。

解决方法：pBuf = create_string_buffer(m_frame.uilmgSize)

pointerData = c_void_p(m_frame.pBuffer + m_frame.usHeader)

memmove(pBuf, pointerData, m_frame.uilmgSize)

buffers = bytes(pBuf)

size = m_frame.usWidth * m_frame.usHeight

for i in range(size):

value = buffers[i*2] + buffers[i*2+1] * 256, 即 value 就是获取到灰度值。

问题 8：设置曝光时间返回成功了但是数据流上没有效果。

问题原因：开启了自动曝光导致手动曝光设置无效。

解决方法：先将自动曝光接口设置为关闭状态，然后再来设置曝光时间即可生效。

问题 9：二次开发亮度和我们软件显示的亮度不一致问题。

问题原因：我们软件是显示有效位的高八位数据，还有曝光、增益、色阶、伽马、对比度、锐度等功能参数是否一致。

解决方法：相机功能参数值需要设置一致，界面显示数据有效位需要一致。



问题 10: TUCAM_Buf_WaitForFrame 函数获取图像超时失败。

问题原因: 接口获取数据超时退出了。

解决方法: 可以将超时时间适当的设大。